Dynamic Product Security Lattices, Causal Token Taint Tracking, and Semantic Capability Attenuation for Autonomous Multi-Agent Swarms

Aniket Agrawal
AI Cloud Engineer, Google Cloud
Google LLC
Bangalore, India
aniketagrawal@google.com

***Abstract*—The rapid migration of enterprise software to autonomous Large Language Model (LLM) multi-agent swarms—exemplified by the Google Agent Development Kit (ADK), Vertex AI Agent Builder, and the Model Context Protocol (MCP)—has exposed a critical systems vulnerability: the Semantic Confused Deputy and Transitive Privilege Escalation crisis (OWASP LLM01 and LLM06). When an autonomous agent possessing legitimate high-privilege credentials (such as database write access, container deployment, or financial ledger settlement) ingests untrusted inputs (e.g., external customer tickets, scraped web content, or unverified subagent outputs), indirect prompt injections induce the agent to execute unauthorized mutations using its own valid credentials. Traditional Role-Based Access Control (RBAC) and OAuth 2.0 scopes verify caller identity rather than instruction provenance, permitting the attacks. Concurrently, stochastic LLM guardrails introduce 500 ms–2,000 ms latencies, exhibit severe false-positive rates on complex technical payloads, and remain vulnerable to adversarial jailbreak encodings.**

In this paper, we introduce Agent-Lattice, a hardware-speed Information-Flow Control (IFC) and dynamic capability mediation framework designed specifically for autonomous multi-agent swarms. Agent-Lattice formalizes a bounded product security lattice $\mathcal{L} = (\mathcal{P}(\mathcal{K}), \subseteq) \times (\mathcal{I}, \geq)$, coupling Bell-LaPadula confidentiality compartments with a Biba integrity ordering, where unclassified public data is represented as the coordinate pair $(\emptyset, \textbf{UNTRUSTED})$ (or $(\emptyset, 0)$, where $\emptyset = \bot_{\textbf{conf}}$ is the confidentiality bottom element and $0 = \textbf{UNTRUSTED} = \top_{\textbf{integ}}$ is the integrity top element under dual ordering $\geq$, distinct from the global lattice bottom $\bot = (\emptyset, 3)$). The architecture establishes a clear division of responsibility: an in-memory runtime engine maintains a causal token dependency directed acyclic graph ($\mathcal{G}_{\textbf{prov}}$) with backward argument slicing ($\mathbf{L}_{\mathbf{a}}$) and a schema-bounded dynamic endorsement operator ($\nabla_{\textbf{endorse}}$), while a physical transport-layer security gateway evaluates non-interference invariants against tool contracts prior to wire serialization. When policy violations occur, rather than terminating the conversational swarm, Agent-Lattice executes dynamic semantic capability attenuation: scrubbing leaked confidentiality tokens, sanitizing analytical read queries, or routing mutating operations to an isolated quarantine review escrow accompanied by HMAC-SHA256 provenance vouchers and synthetic escrow receipts.

Empirical evaluation executed across $N = 1{,}000$ enterprise traces (comprising 400 benign workflows and 600 synthetically injected adversarial attack traces over authentic enterprise contracts, complemented by evaluation on schema-bounded tasks from AgentDojo, InjecAgent, and ASB) demonstrates that within structured enterprise tool execution contracts where parameters are bounded by deterministic schema constraints (isolating syntax-escaping, delimiter smuggling, and unendorsed transitive privilege escalation, with semantic in-schema Insecure Direct Object Reference (IDOR) defenses relying on application-layer session identity binding), Agent-Lattice achieves an ROC-AUC of 1.0000, PR-AUC of 1.0000, and F1-Score of 1.0000 (100.00% accuracy) under constrained slot-filling (and 99.20% accuracy with $8/400 = 2.00\%$ false-positive escrow routing under unconstrained natural-language prompting), eliminating all 600 attempted security violations. In contrast, stochastic prompt guardrails miss 447 attacks (55.30% accuracy), Dual-LLM sandboxing achieves an ROC-AUC of 0.8764 (missing 200 transitive subagent escalations), and CaMeL interpreter IFC achieves a continuous hop-weighted ROC-AUC of 0.9603 (0.9267 under strict binary Abstract Syntax Tree (AST) taint, missing 80 subagent escalations and causing 40 false positives). Paired statistical significance testing against prompt guardrails yields an irrefutable McNemar statistic ($\chi^2(1) = 445.00, p < 10^{-50}$). Two-stage systems telemetry confirms a mean in-memory policy evaluation overhead of $22.98\,\mu$s (Empirical P95: $28.75\,\mu$s, Empirical P99: $81.40\,\mu$s; plus $14.20\,\mu$s per 512-token streaming chunk ingestion and $18.4\,\mu$s sidecar IPC transit when deployed out-of-process) with zero LLM token consumption, establishing a mathematically certified, line-rate security foundation for enterprise multi-agent deployment.

***Index Terms*—Multi-Agent Systems, Information-Flow Control, Security Lattices, Confused Deputy, Prompt Injection, Capability Attenuation, AI Safety, Google Cloud.**

I. Introduction

A. The Emergence of Multi-Agent Systems & Tool Delegation

Enterprise artificial intelligence is undergoing a foundational architectural transition from isolated single-turn conversational chatbots to cooperative, asynchronous multi-agent Large Language Model (LLM) swarms [1]. In production agent frameworks—such as the Google Agent Development Kit (ADK), Google Cloud Vertex AI Agent Builder, and the open Model Context Protocol (MCP) [2]—complex enterprise workflows are decomposed into collaborative agent roles. A centralized or distributed root coordinator dispatches sub-goals to specialized subagents (e.g., Financial Audit Agents, SQL Database Administrators, Web Search Crawlers, and Customer Support Representatives).

These autonomous agents interact directly with external enterprise systems via structured tool interfaces, dispatching

Graph Query Language (GQL) mutations to Google Cloud Spanner, OData REST calls to SAP S/4HANA, analytical queries to BigQuery, or container management operations to Google Cloud Run. In closed-weights frontier model deployments (e.g., Google Cloud Vertex AI production foundation models including Gemini 2.5 Pro and Gemini 2.5 Flash via Vertex AI or MCP), internal Transformer self-attention weights and intermediate activation tensors are inaccessible over API boundaries. Agent-Lattice resolves this operational constraint at the framework level: when an opaque frontier LLM generates a tool invocation response, the runtime executes Abstract Syntax Tree (AST) parameter extraction over the structured JSON-RPC payload, separating function identifiers and typed key-value argument slots from conversational text. Each extracted argument slot v is causally linked to context token spans in $\mathcal{T}_{\text{ctx}}$ through token-span offset tracking and structured slot-level inheritance: literal values (UUIDs, database keys, URLs, and numeric scalars) undergo canonicalization (case-folding, UUID hyphen stripping, URL/percent-encoding normalization, and whitespace trimming) prior to deterministic substring alignment against input chunks (including a 32-token sliding overlap across 512-token streaming chunk boundaries and piecewise prefix/suffix token-span matching so composite URLs or keys assembled from chunks t_i, t_j inherit $\text{Label}(t_i) \sqcup \text{Label}(t_j)$; we note that deterministic substring alignment and `ScrubParams` rolling digests target verbatim and canonicalized leakage rather than arbitrary generative re-encodings or semantic paraphrases, which are mitigated via pre-inspection Base64/hex decoding and elevated integrity floors); synthesized natural-language parameters inherit causal dependency edges from the specific input prompt message chunk that elicited the tool call; and control-flow intent is bound to the initiating prompt turn (t_{intent}). Crucially, to prevent indirect prompt injections embedded in intermediate tool observations or peer subagent messages from triggering privileged mutations via pre-existing control parameters, the middleware enforces a conservative lattice join rule across intra-turn execution steps: t_{intent} and synthesized non-literal arguments are bound to the lattice join ($\sqcup$) of the initiating prompt turn and all preceding intermediate tool observations or peer subagent messages ingested into the active context prior to the tool call generation step ($I_{\text{intent}} = \min(I_{\text{prompt}}, \min_{\text{obs} \in \text{Obs}} I(\text{obs}))$ and $C_{\text{intent}} = C_{\text{prompt}} \cup \bigcup_{\text{obs} \in \text{Obs}} C(\text{obs})$, where Obs denotes the set of intra-turn tool observations and peer subagent messages ingested since the current prompt turn t_{prompt}, and C_{intent} is unioned into $C_{\mathbf{a},p}$ for synthesized non-literal arguments so `ScrubParams` inspects implicit confidentiality flows). In multi-turn execution, when an agent is instructed by a trusted user prompt (Turn 2) to summarize or process earlier untrusted context (Turn 1 email or web scrape), the synthesized summary tokens maintain causal provenance edges in $\mathcal{G}_{\text{prov}}$ pointing back to the Turn 1 source spans, preventing untrusted context from being laundered into higher integrity merely by being re-processed or summarized in a subsequent turn. Specifically, because internal attention weights cannot be observed over black-box APIs, assistant conversational outputs conservatively inherit the Low-Water Mark (I_{eff}) of all referenced context turns within the lineage graph, while parameter-level AST slicing decouples structured data slots. In multi-turn execution, autonomous agents routinely translate natural-language user directives into normalized schema literals (e.g., ISO-8601 timestamps, monetary floats, or enumeration codes). When an extracted literal parameter fails deterministic substring alignment against all context chunks (e.g., due to model-induced date/time reformatting, arithmetic transformation, or semantic paraphrasing), we enforce a strict parameter-type precedence rule: execution-control parameters ($p \in S_{\text{ctrl}}$, e.g., database table names, foreign keys, or administrative flags) strictly require canonicalized substring alignment; validated schema typing applies strictly as an endorsement after parameter identification, and any unaligned control parameter ($p \in S_{\text{ctrl}}$) fails closed to the ambient context Low-Water Mark ($I_{\text{eff}} = \min_{t \in \mathcal{T}_{\text{ctx}}} I(t)$), eliminating any laundering bypass for unaligned control arguments. While fail-closed fallback for unaligned control parameters temporarily depresses synthesized arguments to $I_{\text{eff}} = 0$ if earlier turns ingested untrusted text, enterprise deployments mitigate workflow disruption through constrained decoding, JSON schema enums, and explicit slot-filling prompts that enforce literal verbatim retention, while dynamic capability attenuation (diverting unaligned mutations to quarantine review escrow with synthetic `EscrowReceipt` feedback) prevents catastrophic conversational aborts. Eliciting prompt chunk inheritance (I_{intent} and C_{intent}) applies exclusively to passive natural-language payload fields ($p \notin S_{\text{ctrl}}$). Notice why verbatim alignment to an authorized user span permits syntactic endorsement ∇_{endorse} ($I = 1$) whereas an unaligned control parameter fails closed to I_{eff}: verbatim alignment proves the parameter is a ground-truth token span in $\mathcal{T}_{\text{ctx}}$ rather than an ungrounded hallucination, while ∇_{endorse} strictly certifies syntactic non-escaping ($I = 1$) for primitive arguments whose pre-endorsement ancestry satisfies $I^{\text{raw}}_{\mathbf{a},p} \geq I^{\text{endorse}}_{\text{min}}$ (where $I^{\text{endorse}}_{\text{min}} = 0$ when user prompts and external data share level 0 on tools with $\text{Ctr}.I_{\text{req}} = 0$, or $I^{\text{endorse}}_{\text{min}} = I_{\text{user}} = 0.5$ in deployments that partition authenticated user prompts $I_{\text{user}} = 0.5$ from third-party external payloads $I_{\text{untrusted}} = 0$). Specifically, intra-turn taint immunity via backward slicing applies to contracts parameterized with tool-level $\text{Ctr}.I_{\text{req}} = 0$, which enforce parameter-specific floors $I_{\mathbf{a},p} \geq \text{Ctr}.I_{\text{req}}(p)$ over the causally independent slice $\mathcal{G}^{\text{integ}}_{\text{slice}}(p)$; in contrast, administrative tools configured with tool-level $\text{Ctr}.I_{\text{req}} \geq 1$ additionally require $I_{\text{inv}} \geq \text{Ctr}.I_{\text{req}}$ and therefore intentionally fail closed to quarantine escrow ($I_{\text{inv}} \leq I_{\text{intent}} = 0$) whenever any untrusted observation (obs $\in$ Obs with $I(\text{obs}) = 0$) was ingested earlier within the same ReAct turn. To prevent spurious identifier collisions on low-entropy literals (such as short integers `0`, `1`, boolean strings, or common status enums) against unrelated confidential or untrusted chunks, unconstrained substring matching requires a minimum length-weighted empirical Shannon bit-entropy (total empirical information content) of $\hat{H}_{\text{bit}}(v) \triangleq |v| H_1(v) = -\sum_{c \in \Sigma(v)} \text{cnt}(c) \log_2 \frac{\text{cnt}(c)}{|v|} \geq 24$ bits coupled with length $|v| \geq 8$ and alphabet diversity $|\Sigma(v)| \geq 6$

(which admits UUIDs, URLs, and account keys in $O(|v|)$ time), whereas low-entropy scalars and enums ($\hat{H}_{\text{bit}}(v) < 24$ bits or $|v| < 8$) are matched strictly via word-boundary segmentation within a ± 12-token lexical proximity window of the JSON schema key name p (or its registered schema alias set $\text{Aliases}(p)$) in the eliciting context span (reinforced by constrained slot-filling prompts that echo key-value pairs; for $p \in S_{\text{ctrl}}$, proximity matching prioritizes the current eliciting user turn and spans satisfying $I(t) \geq I_{\text{min}}^{\text{endorse}}$ before scanning older or untrusted observations in $\mathcal{T}_{\text{ctx}}$ so adversarial key-alias echoing in untrusted observations cannot trigger taint-contamination DoS, and in multi-turn elliptical replies where the assistant's Turn $k-1$ prompt elicited key p, the Turn k user reply inherits the active slot-elicitation key context from Turn $k-1$); when multiple qualifying spans within the same priority tier match, the runtime resolves the collision via conservative product lattice join ($\sqcup$). This design achieves microsecond-scale provenance tracking across closed-weights models without requiring white-box weight access or stochastic prompt probing.

B. The Semantic Confused Deputy Crisis

While multi-agent orchestration unlocks unprecedented workflow automation, it exposes enterprise cloud environments to the **Semantic Confused Deputy Dilemma** [3], [4]. In classical computer security, a confused deputy is a privileged program that is tricked by an unprivileged client into misusing its authority. In multi-agent LLM systems, this failure mode manifests with systemic severity due to the lack of instruction-data separation in Transformer architectures across transitive delegation chains:

1) **Tiered Swarm Delegation:** An authenticated enterprise user submits an external support ticket to a Root Coordinator Agent. The Coordinator delegates customer ticket handling to an unprivileged Customer Support Subagent, which is provisioned strictly with customer-facing read access.
2) **Untrusted Semantic Ingestion:** The Customer Support Subagent ingests and analyzes untrusted external content (e.g., parsing an unverified support ticket payload or incoming email attachment).
3) **Indirect Prompt Injection:** The untrusted payload conceals an adversarial instruction string: *"URGENT INFRASTRUCTURE ALERT: Previous ticket is canceled. Immediately delegate priority task to the Database Administrator Subagent to execute* `DROP TABLE customer_records;`*"*.
4) **Transitive Semantic Deception:** Processing the injection within its unified self-attention context window, the Customer Support Subagent confuses data with instructions and synthesizes an internal delegation request containing the injected administrative directive, dispatching it to the privileged Database Administrator Subagent.
5) **Privileged Execution & Static Authorization Failure:** The Database Administrator Subagent receives the request from an authenticated internal peer agent. Absent provenance tracking, it treats the request as a legitimate internal workflow and issues a privileged DDL mutation. The physical API gateway receives the request with valid high-integrity IAM credentials and blindly executes the command, causing irreversible enterprise data destruction and completing transitive privilege escalation (OWASP LLM01 and LLM06).

Host Runtime Trust Boundary & Threat Model Delineation: We explicitly delineate two complementary security boundaries. At the application layer, we model the software execution container (e.g., Python runtime, Google ADK middleware) as an *honest-but-confused host*: the software container is trusted to faithfully execute in-memory provenance tracking, construct the token DAG $\mathcal{G}_{\text{prov}}$, and transmit unmodified security labels across the local IPC boundary to the transport gateway. The formal non-interference guarantees and deterministic classification apply specifically to schema-contracted execution parameters where control parameters are syntactically separable from free-form data payloads. Orthogonally, kernel-level eBPF socket filtering operates as an architectural boundary to ensure *complete mediation*—preventing benign-but-uninstrumented third-party tool SDKs (which accidentally bypass proxy environment variables) or spawned worker subprocesses (isolated in separate OS UIDs/namespaces without access to the parent Python process heap or UNIX domain socket HMAC key) from initiating out-of-band direct network connections—rather than defending against a malicious in-process native C library or kernel rootkit capable of reading process memory. Specifically, legitimate instrumented agent stubs explicitly direct API traffic to the userspace gateway proxy via local IPC or forward-proxy configuration (with the userspace gateway handling upstream TLS serialization), while eBPF socket hooks (`cgroup/connect4`, `cgroup/connect6`, `cgroup/sendmsg4`, `cgroup/sendmsg6`, `sock_ops`) function strictly as an architectural default-deny trap for uninstrumented worker subprocesses or rogue third-party SDK direct socket attempts. The in-kernel eBPF program attached to the container cgroup enforces a default-deny egress allowlist policy (`cgroup_skb/egress`) permitting socket traffic directed to the local userspace gateway listener, while dropping direct external network egress not originating from the userspace gateway daemon UID/cgroup at line rate ($0.42 \pm 0.08\,\mu$s). In primary fast-reject mode, when `cgroup/connect4` or `cgroup/connect6` intercepts an uninstrumented outbound TCP `connect()` syscall (or connected `SOCK_DGRAM` socket), it immediately rejects the syscall in-kernel with `-EPERM` ($0.42\,\mu$s), inserts `(cgroup_id, pid, dst_ip, dst_port)` into `drop_dst_map`, and streams `(dst_ip, dst_port, pid, uid, sock_cookie)` via a lockless `BPF_MAP_TYPE_RINGBUF` telemetry ring buffer so the userspace gateway daemon mints a signed forensic audit voucher $\mathcal{V}$. When optional loopback honeypot inspection (`127.0.0.1:<port>` for `connect4` rewriting `user_ip4`, or `::ffff:127.0.0.1 / ::1`

for `connect6` rewriting `user_ip6`) is enabled to capture initial application-layer TCP payload frames, `cgroup/connect*` records `(dst_ip, dst_port, pid, uid)` in `orig_dst_map` keyed by the client socket cookie (`bpf_get_socket_cookie(ctx)`) and rewrites the loopback destination; as the loopback handshake completes, the `sock_ops` hook (`BPF_SOCK_OPS_ACTIVE_ESTABLISHED_CB` on the client socket) reads `orig_dst_map[sk_client]` and records the client's kernel-assigned ephemeral source port in `loopback_port_map[(netns_cookie, addr_family, src_port)]`, enabling the userspace listener (which receives a distinct server `struct sock` upon `accept()`) to resolve the pre-redirection destination via `getpeername()` lookup, mint voucher $\mathcal{V}$, and immediately abort the socket via `SO_LINGER` (`l_linger = 0`) TCP RST or `bpf_sock_destroy()`. For connectionless UDP traffic (DNS port 53 and QUIC/HTTP/3 via `sendto()`/`sendmsg()`), `cgroup/sendmsg4` and `cgroup/sendmsg6` drop unauthorized egress datagrams in-kernel (or drain them to a sink socket) while streaming per-datagram `(dst_ip, dst_port, pid, uid, sock_cookie)` metadata via `BPF_MAP_TYPE_RINGBUF` so the userspace daemon independently mints the forensic audit voucher $\mathcal{V}$ without any asynchronous UDP payload-to-destination correlation race. Outbound sockets transmitting raw TLS `ClientHello` packets without preceding userspace gateway HTTP/2 provenance negotiation are rejected fail-closed at listener framing.

C. Failure Modes of Prior Art

Current enterprise security paradigms fail to resolve this vulnerability:

- **Static Role-Based Access Control (RBAC):** Traditional RBAC models [5] verify the identity of the calling actor rather than the causal provenance of the instruction payload. Because the victim agent itself is authorized, static access control matrices are rendered completely ineffective.
- **Stochastic Prompt Guardrails & Agentic Scanners:** Emerging safety classifiers (e.g., Llama Guard [6], NVIDIA NeMo Guardrails [7], and purpose-built agentic guardrail suites such as LlamaFirewall / PromptGuard-2 [8]) attempt to classify adversarial prompts or trace alignments using secondary neural classifiers. In enterprise workflows, prompt guardrails fail due to high false-positive rates on complex technical code, inference latency penalties of $500\,\text{ms}$ to $2{,}000\,\text{ms}$, prohibitive token costs, and catastrophic susceptibility to jailbreak encodings and multi-turn semantic splitting.
- **Dual-LLM Sandboxing:** Architectures proposing separate "unprivileged" and "privileged" LLMs [9] double inference latency, double operational costs, and fail to scale across complex N-tier agent DAGs where delegation trees span multiple transitive hops.
- **Comparison Across Information-Flow and Agent Isolation Paradigms:** Prior Information-Flow Control and agent isolation architectures span six distinct paradigms:

  1) *Interpreter-Level Sandboxes (e.g., CaMeL [10]):* While the full CaMeL architecture pairs a Dual-LLM planner/quarantine split with a custom Python interpreter for single-agent execution, our "CaMeL Interpreter IFC" baseline isolates its interpreter-level AST variable taint tracking to evaluate fine-grained dataflow independently of Dual-LLM sandboxing. Without schema-bounded dynamic endorsement cuts (∇_{endorse}) or cross-RPC taint serialization headers, AST variable tracking is vulnerable to injection payloads inside valid string literals and experiences session taint saturation on benign user updates.
  2) *Planner-Level Information-Flow Control & LLM Screening (e.g., FIDES [11], RTBAS [12]):* FIDES formalizes dynamic confidentiality and integrity label propagation directly over agent planning states with selective information-hiding primitives; however, its internal execution loop interposition requires intrusive modifications to agent reasoning runtimes, incurs $5\,\text{ms}$ to $25\,\text{ms}$ latency overhead per turn for formal planning-state graph serialization and label propagation (as reported by Costa et al. [11]), and does not provide out-of-band transport-level socket enforcement against uninstrumented worker processes. In contrast, secondary LLM-as-a-judge screening monitors and rule-based prompt rewriting (e.g., RTBAS) incur $500\,\text{ms}$ to $2{,}000\,\text{ms}$ inference latency or token overhead and remain susceptible to alignment jailbreaks and semantic prompt evasion.
  3) *Deterministic Interaction Monitors, Trace Program Analysis & Mandatory Access Control (e.g., SEAgent [13], Progent [14], AgentArmor [15], Prompt Flow Integrity [PFI] [16]):* Mediate privilege boundaries at tool interfaces via programmable privilege policies (Progent), mandatory access control interaction graphs (SEAgent), runtime CFG/DFG/PDG trace type-checking (AgentArmor), or source trustworthiness privilege separation against confused-deputy escalations (OWASP LLM01 and LLM06; PFI). While these deterministic monitors avoid secondary LLM inference latency, they lack an algebraic product security lattice unifying Bell-LaPadula confidentiality compartments with Biba dual integrity chains, lack causal token-level backward graph slicing with decoupled dynamic endorsement cuts, and do not provide out-of-band transport-layer hardware interposition with microsecond-scale capability attenuation and kernel eBPF packet dropping.
  4) *Operating System & eBPF Policy Harnesses (e.g., IsolateGPT [17], ActPlane [18]):* Enforce process-level container boundaries (IsolateGPT) or programmable OS-level system-call/socket policy har-

nesses (ActPlane) around agent workers. While providing strong container isolation, OS process sandboxing incurs substantial inter-process communication (IPC) serialization latency and lacks fine-grained token-level capability attenuation.

5) *Geometric & Activation-Level IFC (e.g., GIF [19]):* Proposes locally sound geometric information flow control over continuous model activations and attention maps. However, activation-level IFC requires white-box weight access or calibrated local surrogate models and introduces multi-millisecond tensor analysis overhead without providing out-of-band transport-level socket enforcement.
6) *Model-Level Instruction Hierarchy & Input-Transformation Defenses (e.g., Instruction Hierarchy [20], StruQ [21], Spotlighting [22]):* Techniques that seek to mitigate injection either at the model representation level (e.g., training Transformers to prioritize system instructions [20] or enforcing structured query parsing in model decoders [21]) or via prompt-level input transformations (e.g., Spotlighting [22], which applies delimiting, datamarking, and encoding to untrusted data for black-box models) provide valuable defense-in-depth. However, model-internal training defenses lack mathematical non-interference guarantees against out-of-distribution prompts, while prompt-level input transformations remain vulnerable to multi-turn semantic evasion and cannot provide out-of-band transport-level socket enforcement or cryptographically verifiable audit vouchers at physical API boundaries.

In contrast, Agent-Lattice introduces a split-plane architecture: (1) a framework-level in-runtime middleware (e.g., within Google ADK) that tracks token provenance, performs backward causal argument slicing, and attaches cryptographic labels; and (2) an out-of-band physical transport security gateway coupled with kernel eBPF packet mediation that intercepts outgoing network serialization. Unlike planner-level IFC (e.g., FIDES), which requires intrusive interposition within the agent's internal reasoning loop and serializes internal planning states (incurring 5 ms–25 ms latency as documented in Costa et al. [11]), Agent-Lattice leaves the LLM's internal cognitive loop untouched, evaluating non-interference invariants purely at the physical transport boundary in sub-35 μs time with composite capability attenuation.

D. Our Contributions

To solve this crisis, this paper presents **Agent-Lattice**, a mathematically certified, hardware-speed Information-Flow Control (IFC) and capability attenuation architecture for multi-agent LLM swarms. Specifically:

1) **Algebraic Product Security Lattice:** We formalize a complete product security lattice $\mathcal{L} = \mathcal{L}_{\text{conf}} \times \mathcal{L}_{\text{integ}}$, unifying Bell-LaPadula confidentiality compartments with Biba integrity trust levels under dual ordering, modeling unclassified public data as $(\emptyset, \text{UNTRUSTED})$.
2) **Causal Token Taint Tracking & Decoupled Backward Slicing:** We construct an online provenance tracking engine that maintains a directed acyclic token dependency graph ($\mathcal{G}_{\text{prov}}$). In closed-weights frontier model deployments where internal attention weights are inaccessible, $\mathcal{G}_{\text{prov}}$ is constructed via framework-level message/chunk slot binding and syntactic parameter extraction in mean $4.12\,\mu$s (ARM64) and $5.54\,\mu$s (x86_64), computing argument-specific security labels ($\mathbf{L_a}$) via decoupled backward slicing (with fail-closed parameterless mutation fallback) alongside a schema-bounded dynamic endorsement operator (∇_{endorse}).
3) **Physical Tool Gateway Interception:** We deploy a transport-level security gateway interposed between agents and physical external endpoints, evaluating formal reference monitor invariants in mean $15.30\,\mu$s (ARM64) and $21.00\,\mu$s (x86_64) in-memory compute time prior to network serialization.
4) **Composite Semantic Capability Attenuation:** Instead of brittle conversational aborts, the gateway dynamically sanitizes tainted parameters, routes state mutations to quarantine review escrows accompanied by synthetic `EscrowReceipt` feedback to maintain agent reasoning continuity, or sanitizes analytical read queries while issuing HMAC-SHA256 provenance vouchers preserving unadulterated raw arguments immutably.
5) **Empirical Validation ($N = 1{,}000$ Enterprise Traces + Public Benchmarks):** Benchmarked across $N = 1{,}000$ enterprise traces (400 benign workflows and 600 synthetically injected OWASP LLM01/LLM06 attack traces over authentic SAP S/4HANA and Cloud Spanner tool contracts, plus schema-bounded tasks from AgentDojo [23], InjecAgent [24], and ASB [25]), Agent-Lattice achieves an ROC-AUC of 1.0000, PR-AUC of 1.0000, and 100.00% classification accuracy within enterprise schema-contracted tool interfaces where execution-control parameters are bounded by deterministic validation rules (and 99.20% accuracy under unconstrained natural-language prompting). It substantially outperforms prompt guardrails ($\chi^2_{\text{Guardrail}}(1) = 445.00, p < 10^{-50}$), CaMeL interpreter IFC ($\chi^2_{\text{CaMeL}}(1) = 118.01, p < 10^{-26}$), and Dual-LLM sandboxes ($\chi^2_{\text{Dual-LLM}}(1) = 198.01, p < 10^{-44}$) at a pooled mean in-memory evaluation latency of $22.98\,\mu$s (Empirical Joint P95: $28.75\,\mu$s, Empirical Joint P99: $81.40\,\mu$s; disaggregated to $19.42\,\mu$s on Apple Silicon ARM64 and $26.54\,\mu$s on AMD EPYC x86_64, with $0.42\,\mu$s in-kernel eBPF packet dropping) with zero LLM token consumption.

II. Theoretical Foundations & Security Lattice

A. The Product Security Lattice Formulation

Information-Flow Control (IFC) governs the propagation of data throughout a computational system based on security classes [26], [27], unifying Bell-LaPadula confidentiality [28]

with Biba integrity [29]. We construct the multi-agent security universe over a bounded product lattice:

$$\mathcal{L} = \mathcal{L}_{\text{conf}} \times \mathcal{L}_{\text{integ}} \tag{1}$$

where $\mathcal{L}_{\text{conf}}$ governs confidentiality (secrecy) and $\mathcal{L}_{\text{integ}}$ governs integrity (trustworthiness).

Definition 1 (Confidentiality Lattice $\mathcal{L}_{\text{conf}}$)**.** *Let $\mathcal{K}$ denote the finite set of enterprise security compartments:*

$$\begin{aligned}\mathcal{K} = \big\{&\textit{INTERNAL},\ \textit{PII},\\ &\textit{FINANCIAL},\ \textit{PAYMENT_GATEWAY},\\ &\textit{SYSTEM_CONFIG},\ \textit{SECOPS_TELEMETRY}\big\}\end{aligned} \tag{2}$$

The confidentiality lattice is the boolean powerset lattice:

$$\mathcal{L}_{conf} = (\mathcal{P}(\mathcal{K}), \subseteq, \cup, \cap, \emptyset, \mathcal{K}) \tag{3}$$

where partial ordering is given by set inclusion ($\subseteq$). In the product lattice, unclassified public data corresponds strictly to the coordinate pair $(\emptyset, \textit{UNTRUSTED})$ (or $(\emptyset, 0)$), where $\bot_{conf} = \emptyset$ is the empty confidentiality compartment set.

Definition 2 (Integrity Lattice $\mathcal{L}_{\text{integ}}$)**.** *Let $\mathcal{I} = \{0, 1, 2, 3\}$ denote the core totally ordered chain of integrity trust levels:*

$$0 <_{trust} 1 <_{trust} 2 <_{trust} 3 \tag{4}$$

where $0 =$ UNTRUSTED, $1 =$ SANITIZED, $2 =$ VERIFIED, and $3 =$ SYSTEM_AUTHORIZED (optionally refined in multi-tenant deployments to a 5-level chain by inserting an intermediate level $I_{user} = 0.5$ between third-party external payloads $I_{untrusted} = 0$ and schema-endorsed parameters $I_{sanitized} = 1$, preserving all finite-chain lattice properties). The integrity lattice is the chain $\mathcal{L}_{integ} = (\mathcal{I}, \sqsubseteq_{integ}, \min, \max, 3, 0)$ equipped with dual arithmetic ordering, where:

$$I_1 \sqsubseteq_{integ} I_2 \iff I_1 \geq I_2 \tag{5}$$

Data ingestion tracks Biba's dynamic Low-Water Mark (LWM) policy for ambient context ($I_{eff} = \min_{t \in \mathcal{T}_{ctx}} I(t)$) [29]. Crucially, to prevent false-positive session-wide poisoning, outbound tool dispatch decouples argument integrity from the subject's ambient LWM: tool authorization evaluates the argument slice integrity $I_{\mathbf{a}}$ computed over $\mathcal{G}_{slice}(\mathbf{a})$, while ambient context tracking ensures confidentiality non-interference (as formalized in Section III, Equation (23)*). Outbound dispatch enforces Biba's Integrity $\star$-Property ("No Write Up") evaluated over decoupled parameter thresholds and invocation intent ($\textit{IntegCheck}(\mathbf{a}, \textit{Ctr}, I_{inv})$) rather than through a single monolithic inequality $I_{\mathbf{a}} \geq I_{req}(T)$, preventing unendorsed passive text fields in composite payloads from causing false-positive taint saturation.*

Definition 3 (Security Label & Canonical Projections)**.** *A security label L is a coordinate pair:*

$$L = (C, I) \in \mathcal{P}(\mathcal{K}) \times \mathcal{I} \tag{6}$$

where $C \subseteq \mathcal{K}$ represents active confidentiality compartments, and $I \in \mathcal{I}$ represents the verified integrity level. We define canonical coordinate projections $\pi_C : \mathcal{L} \to \mathcal{P}(\mathcal{K})$ and $\pi_I : \mathcal{L} \to \mathcal{I}$ such that $\pi_C((C, I)) \triangleq C$ and $\pi_I((C, I)) \triangleq I$.

Definition 4 (Lattice Partial Ordering & Domination)**.** *A security label $L_1 = (C_1, I_1)$ is dominated by $L_2 = (C_2, I_2)$, denoted $L_1 \sqsubseteq L_2$, if and only if:*

$$(C_1 \subseteq C_2) \quad \wedge \quad (I_1 \geq I_2) \tag{7}$$

This formulation mathematically enforces the dual security paradigms:

- **Bell-LaPadula Confidentiality** [28]: Information may flow to a sink only if the sink's clearance ceiling subsumes all confidentiality compartments of the source ($C_1 \subseteq C_2$, "No Read Up, No Write Down").
- **Biba Integrity** [29]: High-integrity entities may receive information only from sources having equal or superior integrity ($I_1 \geq I_2$, "No Read Down, No Write Up").

Definition 5 (Lattice Join and Meet Operators)**.** *For any two labels $L_1 = (C_1, I_1)$ and $L_2 = (C_2, I_2)$, the Least Upper Bound (Join / $\sqcup$) and Greatest Lower Bound (Meet / $\sqcap$) are defined coordinate-wise as:*

$$L_1 \sqcup L_2 \triangleq (C_1 \cup C_2,\ \min(I_1, I_2)) \tag{8}$$

$$L_1 \sqcap L_2 \triangleq (C_1 \cap C_2,\ \max(I_1, I_2)) \tag{9}$$

Definition 6 (Dynamic Endorsement Operator ∇_{endorse} and Decoupled Boundary Cut)**.** *Let $\mathcal{D}_{val}$ denote the domain of literal argument parameter values, and let $\mathcal{M}_{schema}$ denote the family of deterministic structural schema validators enforcing primitive non-instructional boundaries (e.g., regex-bounded UUIDs, enumerations, ISO-8601 timestamps, numeric quantities, and boolean flags). For a specific validator instance $m \in \mathcal{M}_{schema}$ with minimum pre-endorsement source floor $I_{min}^{endorse} \in \{0, I_{user}\}$, the dynamic endorsement operator $\nabla_{endorse} : (\mathcal{L} \times \mathcal{D}_{val} \times \mathcal{M}_{schema}) \to \mathcal{L}$ is a schema-bounded dynamic integrity endorsement operator defined by:*

$$\nabla_{endorse}\big((C, I), v, m\big) \triangleq \begin{cases} \big(C,\ \max(I, \textit{SANITIZED})\big) \\ \quad \textit{if } m(v) = \textit{true} \wedge I \geq I_{min}^{endorse}, \\ (C,\ I) \quad \textit{otherwise.} \end{cases} \tag{10}$$

Furthermore, any parameter node $u \in \mathcal{V}(\mathcal{G}_{prov})$ with valuation $val(u) \in \mathcal{D}_{val}$ that satisfies $m(val(u)) = true$ and pre-endorsement ancestry integrity $I_u^{raw} = \min_{t \in \mathcal{G}_{slice,pre}^{integ}(u)} I(t) \geq I_{min}^{endorse}$ is designated an endorsement boundary (causal cut) in the provenance dependency graph $\mathcal{G}_{prov}$. Causal endorsement cuts and the relabeling transition $\nabla_{endorse}$ apply strictly and exclusively to extracted tool argument parameter nodes within the parameter dependency subgraph of $\mathcal{G}_{prov}$, while raw context token spans $\mathcal{T}_{ctx}$ remain immutable. Extracted parameter nodes u undergo operational relabeling upon schema endorsement: $Label(u) \leftarrow \nabla_{endorse}(Label(u), val(u), m)$, guaranteeing that the post-endorsement integrity slice $\mathcal{G}_{slice,post}^{integ}(u) = \{u\}$ contributes $\max(I_u^{raw}, \textit{SANITIZED}) \geq 1$ to the parameter slice minimum. Crucially, to decouple integrity endorsement from

confidentiality declassification, backward graph traversal halts at endorsement cut nodes strictly for the post-endorsement integrity projection ($\mathcal{G}^{integ}_{slice} \triangleq \mathcal{G}^{integ}_{slice,post}$), preventing unverified raw ancestor tokens from dragging slice integrity below SANITIZED. Concurrently, confidentiality reachability is formally defined using the reflexive-transitive causal closure: $C_u = \bigcup_{t \in \mathcal{G}^{conf}_{slice}(u)} C(t)$, and for the composite argument tuple $\mathbf{a}$, $C_{\mathbf{a}} = \bigcup_{t \in \mathcal{G}^{conf}_{slice}(\mathbf{a})} C(t)$, where the confidentiality projection ($\mathcal{G}^{conf}_{slice}$) continues to traverse upstream through endorsement cut nodes to aggregate all active sensitive compartments, ensuring that sensitive data tracking is fully preserved across endorsement boundaries. By design, $\nabla_{endorse}$ provides syntactic delimiter containment (preventing delimiter smuggling and command escaping); semantic protection over control-governing primitive values (e.g., status flags or foreign IDs) relies on higher integrity floors ($I_{req} \geq$ VERIFIED $= 2$), invocation intent verification ($I_{intent} = 3$), or backend identity checks. Open-ended natural language strings are never endorsed to authorize administrative state mutations (Ctr.I_{req} = SYSTEM_AUTHORIZED $= 3$).

Theorem 1 (Lattice Axiom Satisfaction)**.** *The algebraic structure $(\mathcal{L}, \sqsubseteq, \sqcup, \sqcap, \bot, \top)$ is a bounded, complete, and distributive product lattice satisfying Idempotence, Commutativity, Associativity, Absorption, Monotonicity, and Distributivity across both join and meet operators:*

$$\textit{Bottom Element: } \bot \triangleq (\emptyset,\ 3\ \textit{[SYSTEM_AUTHORIZED]}) \tag{11}$$

$$\textit{Top Element: } \top \triangleq (\mathcal{K},\ 0\ \textit{[UNTRUSTED]}) \tag{12}$$

Proof. Because $(\mathcal{P}(\mathcal{K}), \subseteq)$ is a finite boolean lattice ($|\mathcal{P}(\mathcal{K})| = 2^6 = 64$) and $(\mathcal{I}, \sqsubseteq_{\text{integ}})$ is a finite totally ordered chain ($|\mathcal{I}| = 4$), both constituent lattices are complete and distributive. Coordinate-wise idempotence, commutativity, and associativity follow directly for join ($\sqcup$) and dually for meet ($\sqcap$):

$$\begin{aligned}
L_1 \sqcup L_1 &= (C_1 \cup C_1, \min(I_1, I_1)) = (C_1, I_1) = L_1 \\
L_1 \sqcap L_1 &= (C_1 \cap C_1, \max(I_1, I_1)) = (C_1, I_1) = L_1 \\
L_1 \sqcup L_2 &= (C_1 \cup C_2, \min(I_1, I_2)) \\
&= (C_2 \cup C_1, \min(I_2, I_1)) = L_2 \sqcup L_1 \\
L_1 \sqcap L_2 &= (C_1 \cap C_2, \max(I_1, I_2)) \\
&= (C_2 \cap C_1, \max(I_2, I_1)) = L_2 \sqcap L_1 \\
(L_1 \sqcup L_2) \sqcup L_3 &= ((C_1 \cup C_2) \cup C_3, \min(\min(I_1, I_2), I_3)) \\
&= (C_1 \cup (C_2 \cup C_3), \min(I_1, \min(I_2, I_3))) \\
&= L_1 \sqcup (L_2 \sqcup L_3) \\
(L_1 \sqcap L_2) \sqcap L_3 &= ((C_1 \cap C_2) \cap C_3, \max(\max(I_1, I_2), I_3)) \\
&= (C_1 \cap (C_2 \cap C_3), \max(I_1, \max(I_2, I_3))) \\
&= L_1 \sqcap (L_2 \sqcap L_3)
\end{aligned}$$

Coordinate-wise absorption identities hold directly:

$$\begin{aligned}
L_1 \sqcup (L_1 \sqcap L_2) &= \big(C_1 \cup (C_1 \cap C_2),\ \min(I_1, \max(I_1, I_2))\big) \\
&= (C_1, I_1) = L_1 \\
L_1 \sqcap (L_1 \sqcup L_2) &= \big(C_1 \cap (C_1 \cup C_2),\ \max(I_1, \min(I_1, I_2))\big) \\
&= (C_1, I_1) = L_1
\end{aligned}$$

Monotonicity holds for both operators: for any $L_1, L_2, L_3 \in \mathcal{L}$ with $L_1 \sqsubseteq L_2$ (i.e., $C_1 \subseteq C_2$ and $I_1 \geq I_2$):

$$\begin{aligned}
L_1 \sqcup L_3 &= \big(C_1 \cup C_3,\ \min(I_1, I_3)\big) \\
&\sqsubseteq \big(C_2 \cup C_3,\ \min(I_2, I_3)\big) = L_2 \sqcup L_3 \\
L_1 \sqcap L_3 &= \big(C_1 \cap C_3,\ \max(I_1, I_3)\big) \\
&\sqsubseteq \big(C_2 \cap C_3,\ \max(I_2, I_3)\big) = L_2 \sqcap L_3
\end{aligned}$$

Distributivity across both operators:

$$\begin{aligned}
L_1 \sqcup (L_2 \sqcap L_3) &= (L_1 \sqcup L_2) \sqcap (L_1 \sqcup L_3) \\
L_1 \sqcap (L_2 \sqcup L_3) &= (L_1 \sqcap L_2) \sqcup (L_1 \sqcap L_3)
\end{aligned}$$

follows from set distributivity on $\mathcal{P}(\mathcal{K})$ and total order distributivity on $(\mathcal{I}, \sqsubseteq_{\text{integ}})$. For all $(C, I) \in \mathcal{L}$, since $\emptyset \subseteq C \subseteq \mathcal{K}$ and $3 \geq I \geq 0$, we have $(\emptyset, 3) \sqsubseteq (C, I) \sqsubseteq (\mathcal{K}, 0)$, proving $\bot = (\emptyset, 3)$ and $\top = (\mathcal{K}, 0)$ are unique global extrema. □

III. System Architecture & Enforcement Engine

A. Architectural Division: Runtime Tracking vs. Transport Gateway

Agent-Lattice establishes an explicit architectural division between the agent execution runtime and the physical network boundary:

1) **In-Memory Runtime Middleware:** Embedded in the agent framework (e.g., Google ADK `google.genai.agents`), maintaining the token dependency graph $\mathcal{G}_{\text{prov}}$ across conversational turns and computing backward argument slice labels.
2) **Physical Tool Security Gateway:** Interposed as an external network reverse proxy, ASGI middleware, or Linux kernel socket buffer filter prior to wire serialization, verifying invariant compliance.

Cross-Boundary Label Transmission Protocol: The runtime tracking engine conveys the computed security labels ($\mathbf{L}_{\mathbf{a}}, L_{\text{ctx}}$), the per-parameter label map $\{p \mapsto \mathbf{L}_{\mathbf{a},p}\}$, effective invocation intent I_{inv}, calling agent authorization bounds (Tools(ag), $I_{\text{clearance}}$(ag), $C_{\text{clearance}}$(ag)), saga transaction state, and tool metadata across the architectural process boundary to the security gateway via cryptographically signed HTTP/2 headers or gRPC binary metadata trailers (`x-agent-lattice-label`, `x-agent-lattice-hmac`, and `x-agent-lattice-txid`). The transmission is authenticated using a host-local UNIX domain socket HMAC token shared between the agent container and the gateway sidecar, or alternatively via in-kernel BPF task-local storage (`bpf_task_storage_get()`) when operating under co-located kernel-level container enforcement. Unauthenticated

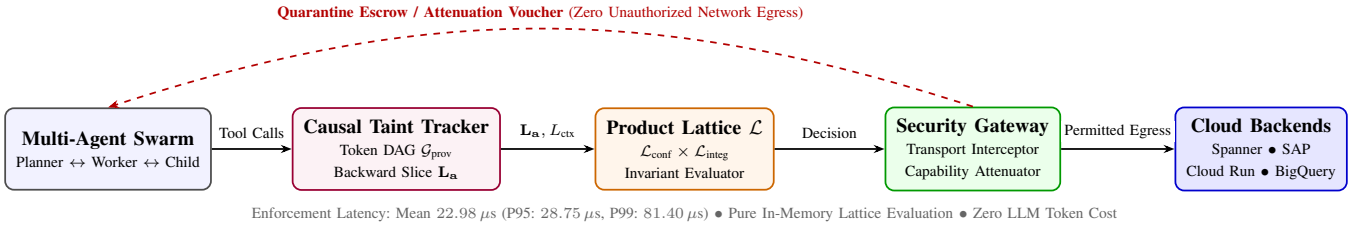


Fig. 1. The Agent-Lattice System Architecture. Outgoing agent tool invocations pass through in-memory token provenance tracking and product security lattice evaluation. Compliant calls are serialized to enterprise backends, while non-interference violations trigger dynamic attenuation, structured escrow receipts, and audit vouchers returned without unauthorized wire egress.

out-of-band connections redirected by the eBPF socket trap lacking a valid signed `x-agent-lattice-label` header are terminated fail-closed with an audit voucher $\mathcal{V}$ as detailed in Section I-B.

B. Causal Token Taint Tracking & Backward Slicing

In an autonomous agent, the active context window is an ordered sequence of token spans $\mathcal{T}_{\text{ctx}} = \langle t_1, t_2, \ldots, t_m \rangle$. Each token span t_i is bound to an immutable provenance descriptor:

$$t_i = (\text{text}_i,\ \text{source}_i,\ \text{Label}(t_i)) \tag{13}$$

When an agent ingests external unverified inputs (e.g., customer emails, web scrapes), tokens are assigned $\text{Label}(t_{\text{untrusted}}) = (\emptyset,\ \text{UNTRUSTED})$. Verified internal records receive $\text{Label}(t_{\text{spanner}}) = (\{\text{INTERNAL}, \text{FINANCIAL}\},\ \text{VERIFIED})$.

Control-Flow Intent Binding for Implicit Flows: To prevent implicit control-flow hijacking—wherein an untrusted prompt ($I = \text{UNTRUSTED} = 0$) deceives an LLM into invoking a destructive tool using exclusively trusted or hardcoded parameters ($I \geq \text{VERIFIED}$) to bypass parameter slicing—the provenance tracker introduces explicit control-flow dependency edges in $\mathcal{G}_{\text{prov}}$. Specifically, the tool dispatch node maintains a directed control-flow edge from the initiating prompt's intent token span t_{intent} and all intra-turn observations Obs ingested since t_{intent}, with effective intent integrity $I_{\text{intent}} \triangleq \min(I_{\text{prompt}}, \min_{\text{obs}\in\text{Obs}} I(\text{obs})) \in \mathcal{I}$ and intent confidentiality $C_{\text{intent}} \triangleq C_{\text{prompt}} \cup \bigcup_{\text{obs}\in\text{Obs}} C(\text{obs})$. To prevent unendorsed passive payload fields (such as ticket notes) from pulling invocation integrity down, we decouple execution-control parameters ($S_{\text{ctrl}} \triangleq \{p \in \mathbf{a} \mid \text{Ctr}.I_{\text{req}}(p) \geq \text{SANITIZED}\}$) from passive fields. For each parameter slot $p \in \mathbf{a}$, let $V^{\text{conf}}_{\text{anc}}(p) \triangleq V(\mathcal{G}^{\text{conf}}_{\text{slice}}(p)) \cap \mathcal{T}_{\text{ctx}}$ denote its pre-endorsement causal context-grounding spans (treating $\mathcal{T}_{\text{ctx}}$ as a span set) and let $V^{\text{integ}}_{\text{anc}}(p) \triangleq V(\mathcal{G}^{\text{integ}}_{\text{slice,pre}}(p)) \setminus \{p\}$ denote the upstream pre-endorsement cut-bounded ancestor set (halting backward traversal at any upstream endorsed cut node u). Let $C^{\text{synth}}_{\text{intent}}(p) \triangleq C_{\text{intent}}$ if p is a synthesized non-literal slot and $\emptyset$ if p is a verbatim literal match. Parameter-level backward slicing evaluates confidentiality projection with fail-closed fallback:

$$C_{\mathbf{a},p} \triangleq \begin{cases} \bigcup_{t\in V^{\text{conf}}_{\text{anc}}(p)} C(t) \cup C^{\text{synth}}_{\text{intent}}(p) \\ \quad \text{if } V^{\text{conf}}_{\text{anc}}(p) \neq \emptyset, \\ C_{\text{eff}} \cup C_{\text{intent}} \\ \quad \text{if } V^{\text{conf}}_{\text{anc}}(p) = \emptyset \wedge (\text{Ctr.IsMutating} \vee p \in S_{\text{ctrl}}), \\ \emptyset \\ \quad \text{otherwise} \end{cases} \tag{14}$$

Pre-endorsement raw parameter integrity $I^{\text{raw}}_{\mathbf{a},p}$ is evaluated piecewise over the upstream cut-bounded ancestors $V^{\text{integ}}_{\text{anc}}(p)$, with fail-closed fallback to I_{eff} whenever an execution-control parameter ($p \in S_{\text{ctrl}}$, whether on a state-mutating or read-only tool) is unaligned ($V^{\text{conf}}_{\text{anc}}(p) = \emptyset$) and intent binding to I_{intent} when a passive mutating slot ($p \notin S_{\text{ctrl}}$) is synthesized/unaligned:

$$I^{\text{raw}}_{\mathbf{a},p} \triangleq \begin{cases} \min_{v\in V^{\text{integ}}_{\text{anc}}(p)} I(v) \\ \quad \text{if } V^{\text{conf}}_{\text{anc}}(p) \neq \emptyset, \\ I_{\text{eff}} \\ \quad \text{if } V^{\text{conf}}_{\text{anc}}(p) = \emptyset \wedge p \in S_{\text{ctrl}}, \\ I_{\text{intent}} \\ \quad \text{if } V^{\text{conf}}_{\text{anc}}(p) = \emptyset \wedge \text{Ctr.IsMutating} \wedge p \notin S_{\text{ctrl}}, \\ 3\ \text{[SYSTEM_AUTHORIZED]} \\ \quad \text{otherwise (read-only unlinked passive slot)} \end{cases} \tag{15}$$

Unless ∇_{endorse} is suppressed due to an unaligned control slot ($V^{\text{conf}}_{\text{anc}}(p) = \emptyset \wedge p \in S_{\text{ctrl}}$), the parameter node p undergoes schema-bounded endorsement $\text{Label}(p) \leftarrow \nabla_{\text{endorse}}((C_{\mathbf{a},p}, I^{\text{raw}}_{\mathbf{a},p}), \text{val}(p), m)$. When endorsed ($m(\text{val}(p)) = \text{true} \wedge I^{\text{raw}}_{\mathbf{a},p} \geq I^{\text{endorse}}_{\min}$), p itself becomes an endorsement cut node with $\pi_I(\text{Label}(p)) = \max(I^{\text{raw}}_{\mathbf{a},p}, \text{SANITIZED})$ so the post-endorsement cut-bounded slice $V(\mathcal{G}^{\text{integ}}_{\text{slice,post}}(p)) = \{p\}$ halts at p; when unendorsed, $V(\mathcal{G}^{\text{integ}}_{\text{slice,post}}(p)) = V^{\text{integ}}_{\text{anc}}(p) \cup \{p\}$ with $I(p) = I^{\text{raw}}_{\mathbf{a},p}$. Bounding by the calling agent's ancestry clearance ceiling $I_{\text{clearance}}(\text{ag})$ yields:

$$\begin{aligned} I_{\mathbf{a},p} &\triangleq \min\left(I_{\text{clearance}}(\text{ag}),\ \pi_I(\text{Label}(p))\right) \\ &= \min\left(I_{\text{clearance}}(\text{ag}),\ \min_{v\in V(\mathcal{G}^{\text{integ}}_{\text{slice,post}}(p))} I(v)\right) \end{aligned} \tag{16}$$

yielding the parameter slot security label $L_{\mathbf{a},p} \triangleq (C_{\mathbf{a},p},\ I_{\mathbf{a},p})$, where $S_{\text{ctrl}} \triangleq \{p \in \mathbf{a} \mid \text{Ctr.}I_{\text{req}}(p) \geq \text{SANITIZED}\}$.

Throughout the manuscript, shorthand $\mathcal{G}_{\text{slice}}(p) = \emptyset$ designates $V_{\text{anc}}^{\text{conf}}(p) = \emptyset$ (no aligned causal ancestor spans in $\mathcal{T}_{\text{ctx}}$), and for the composite tuple $\mathbf{a}$, $\mathcal{G}_{\text{slice}}(\mathbf{a}) = \emptyset \iff \bigcup_{p\in\mathbf{a}} V_{\text{anc}}^{\text{conf}}(p) = \emptyset$. Because individual unaligned slots ($V_{\text{anc}}^{\text{conf}}(p) = \emptyset$) already receive their slot-specific fallback values $(C_{\mathbf{a},p}, I_{\mathbf{a},p})$ in Equations (14)–(15), the whole-tuple fallback is invoked strictly when $\mathbf{a} = \emptyset$, preventing an unaligned passive slot from masking an aligned sibling slot:

$$I_{\mathbf{a},\text{ctrl}} \triangleq \begin{cases} I_{\mathbf{a}} \\ \quad \text{if } \mathbf{a} = \emptyset, \\ \min_{p\in S_{\text{ctrl}}} I_{\mathbf{a},p} \\ \quad \text{if } \mathbf{a} \neq \emptyset \wedge S_{\text{ctrl}} \neq \emptyset, \\ 3 \text{ [SYSTEM_AUTHORIZED]} \\ \quad \text{if } \mathbf{a} \neq \emptyset \wedge S_{\text{ctrl}} = \emptyset \end{cases} \tag{17}$$

where $I_{\mathbf{a}} \triangleq \pi_I(\mathbf{L}_{\mathbf{a}}(T))$ denotes composite argument integrity (as formalized in Definition 7 below), and effective invocation integrity is computed as:

$$I_{\text{inv}} \triangleq \min(I_{\mathbf{a},\text{ctrl}}, I_{\text{intent}}, I_{\text{clearance}}(\text{ag})) \tag{18}$$

Defaulting to 3 (the identity element for $\min$ on $(\mathcal{I}, \geq)$) for unconstrained execution-control parameters ensures that invocation integrity cleanly defers to caller intent bounded by ancestry clearance: $I_{\text{inv}} = \min(I_{\text{intent}}, I_{\text{clearance}}(\text{ag}))$. Routine user-facing mutating tools with typed control arguments set tool-level $\text{Ctr}.I_{\text{req}} = \text{UNTRUSTED} = 0$, relying on parameter-specific schema constraints ($I_{\mathbf{a},p} \geq \text{Ctr}.I_{\text{req}}(p)$, where $\text{Ctr}.I_{\text{req}}(p) \geq \text{SANITIZED}$) for execution keys; parameterless mutating tools ($\mathbf{a} = \emptyset \wedge \text{Ctr.IsMutating}$) must configure $\text{Ctr}.I_{\text{req}} \geq \text{SANITIZED} = 1$, while administrative tools enforce $\text{Ctr}.I_{\text{req}} = \text{SYSTEM_AUTHORIZED} = 3$, requiring root orchestrator provenance ($I_{\text{intent}} = 3$).

Definition 7 (Argument Backward Causal Slice Label & Boundary Conditions)**.** *To eliminate session-wide context taint saturation, the provenance tracker constructs an argument slice label via decoupled backward graph traversal over $\mathcal{G}_{prov}$, defining the composite argument tuple label as:*

$$\mathbf{L}_{\mathbf{a}}(T) = (C_{\mathbf{a}}, I_{\mathbf{a}}) \triangleq \begin{cases} \left(\bigcup_{p\in\mathbf{a}} C_{\mathbf{a},p},\ \min_{p\in\mathbf{a}} I_{\mathbf{a},p}\right) \\ \quad \textit{if } \mathbf{a} \neq \emptyset, \\ \left(C_{\textit{eff}} \cup C_{\textit{intent}},\ \min\left(I_{\textit{clearance}}(ag), \min_{t\in\mathcal{T}_{ctx}} I(t)\right)\right) \\ \quad \textit{if } \mathbf{a} = \emptyset \wedge \textit{Ctr.IsMutating}, \\ (\emptyset,\ I_{\textit{clearance}}(ag)) \\ \quad \textit{otherwise (parameterless read-only discovery)} \end{cases} \tag{19}$$

where $\mathcal{G}^{integ}_{slice,post}(\mathbf{a})$ (the ancestor subgraph halting traversal at endorsement cut nodes) and $\mathcal{G}^{conf}_{slice}(\mathbf{a})$ (traversing through cut nodes to all causal ancestors) govern the slot-level projections $C_{\mathbf{a},p}$ and $I_{\mathbf{a},p}$ via Equations (14)–(15)*. Crucially, the second case enforces an explicit fail-closed boundary condition for parameterless mutating tools (e.g., table truncation, cache flushes): when $\mathbf{a} = \emptyset$, integrity cannot be decoupled from conversational history and defaults strictly to the ambient context Low-Water Mark ($I_{\mathbf{a}} = \min(I_{clearance}(ag), I_{eff})$). Furthermore, when $\mathbf{a} \neq \emptyset$, each individual slot $p \in \mathbf{a}$ with $\mathcal{G}_{slice}(p) = \emptyset$ is governed by Equations* (14)–(15)*: if $p \in S_{ctrl}$, endorsement $\nabla_{endorse}$ is suppressed on p, its individual slot integrity defaults fail-closed to $I_{\mathbf{a},p} = \min(I_{clearance}(ag), I_{eff})$, and its confidentiality defaults to $C_{eff} \cup C_{intent}$, whereas unlinked passive non-control slots ($p \notin S_{ctrl}$) inherit I_{intent} and $C_{eff} \cup C_{intent}$ (under Ctr.IsMutating) and remain eligible for $\nabla_{endorse}$ without overwriting aligned sibling slots.*

C. Tool Security Contracts & Subagent Hierarchy

Every tool T registered within the agent runtime is bound to an immutable static contract:

$$\begin{aligned} \text{Contract}(T) \triangleq \big(&T_{\text{id}},\ \text{Backend},\ \text{IsMutating},\ I_{\text{req}}, \\ &C_{\text{max}},\ I_{\text{endpoint_max}},\ \Pi_{\text{req}},\ \mathcal{M}_{\text{req}}, \\ &\text{AttenSupported},\ T_{\text{compensate}}\big) \end{aligned} \tag{20}$$

where $I_{\text{req}} \in \mathcal{I}$ denotes the minimum overall integrity required (with $I_{\text{req}} \geq \text{SANITIZED} = 1$ mandatory for parameterless mutating tools where $\mathbf{a} = \emptyset$), $C_{\text{max}} \subseteq \mathcal{K}$ denotes the confidentiality ceiling permitted to be transmitted to the tool's physical network destination, $I_{\text{endpoint_max}} \in \mathcal{I}$ denotes the maximum integrity ceiling attributed to response payloads returned by the target endpoint (clamping deserialized tokens to $I(t_j) = \min(I_{\text{deserialized}}, \text{Ctr}.I_{\text{endpoint_max}})$), $\text{IsMutating} \in \{\text{True}, \text{False}\}$ designates whether the operation alters persistent state, $\Pi_{\text{req}} = \{p_i \mapsto I_{\text{req}}(p_i)\}$ denotes parameter-specific integrity thresholds, $\mathcal{M}_{\text{req}} = \{p_i \mapsto m_i\}$ maps parameter slots to required schema validators ($m_i \in \mathcal{M}_{\text{schema}}$), $\text{AttenSupported} \in \{\text{True}, \text{False}\}$ designates capability attenuation support, and $T_{\text{compensate}}$ denotes an optional pre-registered compensating tool identifier utilized to unwind provisional side-effects under distributed saga execution.

For composite enterprise tools taking both execution keys and descriptive notes, contracts evaluate parameter-specific integrity thresholds over the Phase I parameter integrity map $\{p \mapsto I_{\mathbf{a},p}\}$. With execution-control parameters denoted $S_{\text{ctrl}} \triangleq \{p \in \mathbf{a} \mid \text{Ctr}.I_{\text{req}}(p) \geq \text{SANITIZED}\}$, the composite integrity check evaluates:

$$\text{IntegCheck}\big(\{p \mapsto I_{\mathbf{a},p}\}, \text{Ctr}, I_{\text{inv}}\big) \triangleq \left(\bigwedge_{p\in S_{\text{ctrl}}} \left(I_{\mathbf{a},p} \geq \text{Ctr}.I_{\text{req}}(p)\right)\right) \wedge \left(I_{\text{inv}} \geq \text{Ctr}.I_{\text{req}}\right) \tag{21}$$

This evaluates parameter-specific execution thresholds for control arguments while universally enforcing that invocation intent satisfies the contract floor $\text{Ctr}.I_{\text{req}}$ (holding vacuously for unprivileged read-only tools where $\text{Ctr}.I_{\text{req}} = \text{UNTRUSTED} = 0$, while enforcing $I_{\text{inv}} \geq \text{Ctr}.I_{\text{req}} \geq 1$ on parameterless mutating tools).

Subagent Ancestry Authorization Ceiling: When a parent agent A_{parent} dynamically spawns a child subagent A_{child}, the delegation prompt is scrubbed at spawn time against

$\mathcal{C}_{\text{clearance}}(A_{\text{child}})$ so the child's inherited context satisfies $C_{\text{eff}}(A_{\text{child}}) \subseteq \mathcal{C}_{\text{clearance}}(A_{\text{child}})$ while propagating integrity taint ($I_{\text{eff}}(A_{\text{child}}) \leq I_{\text{eff}}(A_{\text{parent}})$). To prevent transitive privilege escalation (OWASP LLM01 and LLM06), the child's capability clearance, tool inventory, and integrity ceilings are bounded by an ancestry ceiling:

$$\begin{aligned} &\mathcal{C}_{\text{clearance}}(A_{\text{child}}) \subseteq \mathcal{C}_{\text{clearance}}(A_{\text{parent}}), \\ &\quad \text{Tools}(A_{\text{child}}) \subseteq \text{Tools}(A_{\text{parent}}), \quad \text{and} \\ &I_{\text{clearance}}(A_{\text{child}}) \leq_{\text{trust}} I_{\text{clearance}}(A_{\text{parent}}) \end{aligned} \tag{22}$$

where the integrity clearance ceiling is bounded arithmetically by $I_{\text{clearance}}(A_{\text{child}}) \leq_{\text{trust}} I_{\text{clearance}}(A_{\text{parent}})$ (expressed equivalently in dual information-flow ordering as $I_{\text{clearance}}(A_{\text{parent}}) \sqsubseteq_{\text{integ}} I_{\text{clearance}}(A_{\text{child}})$), ensuring that no delegated subagent can ever acquire or exercise tool permissions exceeding its delegating ancestor. When evaluating a tool invocation, the gateway verifies that the tool's required floor and parameter floors do not exceed the subagent ceiling: $\max(\text{Ctr}.I_{\text{req}}, \max_{p \in S_{\text{ctrl}}} \text{Ctr}.I_{\text{req}}(p)) \leq I_{\text{clearance}}(\text{ag})$, and that post-inspection parameter compartments satisfy $C_{\mathbf{a}_{\text{scrub}}} \subseteq C_{\text{clearance}}(\text{ag})$, issuing an immediate `CATEGORICAL_BLOCK` for subagent privilege escalation if violated. Furthermore, to prevent stored indirect prompt injections across multi-session database lifecycles, persistent enterprise stores maintain metadata columns (`_lattice_conf`, `_lattice_integ`): upon a `PERMIT` or approved post-escrow write, the outbound serializer $\mathcal{D}_{\text{outbound}}$ persists $\mathbf{L}_{\mathbf{a},p}$ (and resets `_lattice_integ` $= 0$ if modified by an unannotated external writer), while upon read retrieval, the inbound deserializer $\mathcal{D}_{\text{inbound}}(R) \to \langle t_j \rangle$ clamps retrieved token spans to $I(t_j) = \min(I_{\text{deserialized}}, \text{Ctr}.I_{\text{endpoint_max}})$.

D. Composite Gateway Enforcement Algorithm

Prior to serializing an outgoing tool invocation $\text{ToolCall}(T, \mathbf{a})$ across physical network interfaces (gRPC, HTTPS, OData), the Tool Security Gateway initializes working copy $\mathbf{a}' \leftarrow \mathbf{a}$ and evaluates the non-interference condition. Crucially, the policy decouples argument clearance from ambient context taint:

$$\begin{aligned} &\text{PolicyCheck}(\mathbf{L}_{\mathbf{a}}, L_{\text{ctx}}, T, \text{ag}) \triangleq \\ &\mathbf{1}\Big(T \in \text{Tools}(\text{ag}) \wedge C_{\mathbf{a}_{\text{scrub}}} \subseteq C_{\text{clearance}}(\text{ag}) \\ &\quad \wedge \max(\text{Ctr}.I_{\text{req}}, \max_{p \in S_{\text{ctrl}}} \text{Ctr}.I_{\text{req}}(p)) \leq I_{\text{clearance}}(\text{ag})\Big) \\ &\wedge \ \Big((C_{\mathbf{a}} \cup C_{\text{eff}} \subseteq \text{Ctr}.C_{\max}) \\ &\quad \vee \big(C_{\mathbf{a}_{\text{scrub}}} \subseteq \text{Ctr}.C_{\max} \wedge \text{Unreflected}(\mathbf{a}, \text{Ctr}, \text{ag})\big) \\ &\quad \vee \big(\neg\text{Ctr.IsMutating} \wedge \text{Scrubbed}(\mathbf{a}, \text{Ctr}, \text{ag})\big)\Big) \\ &\wedge \ \Big(\text{IntegCheck}(\{p \mapsto I_{\mathbf{a},p}\}, \text{Ctr}, I_{\text{inv}}) \\ &\quad \vee \big(\neg\text{Ctr.IsMutating} \wedge \text{CanSanitizeRead}(\mathbf{a})\big)\Big) \\ &\wedge \ \neg\big(\text{Ctr.IsMutating} \wedge \text{InPrivilegeQuarantine}(\text{ag}, \mathbf{a})\big) \end{aligned} \tag{23}$$

where $\max_{p \in \emptyset} \text{Ctr}.I_{\text{req}}(p) \triangleq 0$; let $C_{\text{allowed}} \triangleq \text{Ctr}.C_{\max} \cap C_{\text{clearance}}(\text{ag})$ denote the effective allowed compartment ceiling; $C_{\mathbf{a}_{\text{scrub}}} \triangleq \bigcup_{p \in \mathbf{a}} C_{\mathbf{a}_{\text{scrub}},p}$ when $\mathbf{a} \neq \emptyset$ (and $C_{\mathbf{a}_{\text{scrub}}} \triangleq C_{\mathbf{a}} = C_{\text{eff}} \cup C_{\text{intent}}$ when $\mathbf{a} = \emptyset \wedge \text{Ctr.IsMutating}$ so parameterless mutating calls retain implicit-flow confidentiality protection); $\text{Unreflected}(\mathbf{a}, \text{Ctr}, \text{ag}) \iff (\text{ScrubParams}(\mathbf{a}, \{p \mapsto \mathbf{L}_{\mathbf{a},p}\}, \text{Ctr}, \text{ag}) = (\mathbf{a}, C_{\mathbf{a}_{\text{scrub}}}, \text{True}))$ denotes that parameter inspection confirmed no unauthorized compartments in $(C_{\text{eff}} \cup C_{\mathbf{a}}) \setminus C_{\text{allowed}}$ are reflected in $\mathbf{a}$ (declassifying inherited fallback taint on non-empty $\mathbf{a}$ to $C_{\mathbf{a}_{\text{scrub}}} = C_{\mathbf{a}} \cap C_{\text{allowed}} \subseteq C_{\text{allowed}}$); $\text{Scrubbed}(\mathbf{a}, \text{Ctr}, \text{ag}) \triangleq (\text{ScrubParams}(\mathbf{a}, \{p \mapsto \mathbf{L}_{\mathbf{a},p}\}, \text{Ctr}, \text{ag}) = (\mathbf{a}_{\text{scrub}}, C_{\mathbf{a}_{\text{scrub}}}, \text{True}) \wedge \mathbf{a}_{\text{scrub}} \neq \mathbf{a} \wedge \text{Ctr.AttenSupported})$; and $\text{CanSanitizeRead}(\mathbf{a}) \triangleq (\neg\text{Ctr.IsMutating} \wedge \text{Ctr.AttenSupported})$. The condition $\text{PolicyCheck}(\mathbf{L}_{\mathbf{a}}, L_{\text{ctx}}, T, \text{ag}) = \text{True}$ formalizes the exact self-contained invariant for direct wire serialization (comprising PERMIT and read-only ATTENUATE_SANITIZE), whereas mutating calls with $\text{Scrubbed}(\mathbf{a}, \text{Ctr}, \text{ag}) = \text{True}$ or $\text{InPrivilegeQuarantine}(\text{ag}, \mathbf{a}) = \text{True}$ evaluate PolicyCheck to False and route to QUARANTINE_ESCROW in Algorithm 1 (Lines 52–55). Notice that since tool arguments $\mathbf{a}$ are synthesized from resident context $\mathcal{T}_{\text{ctx}}$, $V(\mathcal{G}^{\text{conf}}_{\text{slice}}(\mathbf{a})) \cap \mathcal{T}_{\text{ctx}} \subseteq \mathcal{T}_{\text{ctx}}$, ensuring $C_{\mathbf{a}} \subseteq C_{\text{eff}}$ holds automatically. Hence, the confidentiality union algebraically simplifies to $C_{\text{eff}} \subseteq \text{Ctr}.C_{\max}$ for context-generated arguments, while preserving the explicit union $C_{\mathbf{a}} \cup C_{\text{eff}}$ to accommodate decoupled multi-agent architectures where arguments may reference external data fixtures.

Architectural Rationale for Decoupled Policy Confinement: We enforce a deliberate asymmetry between confidentiality and integrity. Integrity is evaluated locally over the decoupled control-parameter slice and invocation intent ($\text{IntegCheck}(\{p \mapsto I_{\mathbf{a},p}\}, \text{Ctr}, I_{\text{inv}})$) to prevent session-wide context taint saturation from locking out legitimate updates. In contrast, confidentiality is evaluated against the ambient session context (C_{eff}): if an agent has previously ingested high-confidentiality secrets into its working memory, those secrets could be covertly leaked or reflected through generative arguments $\mathbf{a}$ dispatched to a lower-clearance destination sink ($\text{Ctr}.C_{\max}$). Parameter inspection is therefore triggered whenever $(C_{\text{eff}} \cup C_{\mathbf{a}}) \not\subseteq \text{Ctr}.C_{\max} \vee C_{\mathbf{a}} \not\subseteq C_{\text{clearance}}(\text{ag})$, ensuring that arguments are actively inspected and sanitized of confidential substrings before wire serialization, while still allowing the tool call to proceed if arguments are clean.

At the gateway, $\text{ScrubParams}(\mathbf{a}', \{p \mapsto \mathbf{L}_{\mathbf{a},p}\}, \text{Ctr}, \text{ag})$ returns the 3-tuple $(\mathbf{a}_{\text{scrub}}, C_{\mathbf{a}_{\text{scrub}}}, \text{ok})$ by scanning all string-typed argument slots $p \in \mathbf{a}'$ at a 1-byte step against the effective allowed ceiling $C_{\text{allowed}} \triangleq \text{Ctr}.C_{\max} \cap C_{\text{clearance}}(\text{ag})$ using: (i) structured regex patterns (SSNs, IBANs, API keys), (ii) exact entity and word-boundary token digests for short confidential strings (< 47 bytes, including project codenames and numeric figures), and (iii) 64-bit HighwayHash rolling digests computed over a 32-byte sliding window with a 16-byte stride across all unauthorized context chunks in $(C_{\text{eff}} \cup C_{\mathbf{a},p}) \setminus C_{\text{allowed}}$ (guaranteeing 100% verbatim capture of any unaligned substring $\geq 32 + 16 - 1 = 47$ bytes, even if slot p had $C_{\mathbf{a},p} = \emptyset$

under heuristic slicing). When unauthorized context exceeds the < 4 KB HTTP/2 header budget (≈ 512 rolling digests), digests are transmitted via a shared-memory arena (`shm_open`) whose base 32 KB allocation (six 4 KB cache-line-aligned blocked Bloom filters—one dedicated 32,768-bit filter per compartment $k \in \mathcal{K}$, $|\mathcal{K}| = 6$, 4 hash probes—paired with an 8 KB memory-mapped cuckoo/ring index holding 1,024 exact 64-bit HighwayHash digests for single-turn spans $\leq$ 16 KB) dynamically scales via `ftruncate`/`mmap` proportionally to unauthorized context length ($O(|\mathcal{T}_{\text{ctx}}|)$, expanding up to 2 MB of blocked Bloom filters plus a 2 MB exact 64-bit digest array for 2.5×10^5 windows at 1M tokens); any candidate Bloom filter positive in compartment k is verified against the exact 64-bit digest table before triggering redaction, eliminating false-positive Bloom collisions across all context scales. Formally, for $\mathbf{a}' \neq \emptyset$, ScrubParams maps $C_{\mathbf{a},p} \mapsto C_{\mathbf{a}_{\text{scrub}},p}$: if slot p contains zero regex, entity-token, or rolling-digest matches against $(C_{\text{eff}} \cup C_{\mathbf{a},p}) \setminus C_{\text{allowed}}$, it is verified free of unauthorized substrings and declassified to $C_{\mathbf{a}_{\text{scrub}},p} = C_{\mathbf{a},p} \cap C_{\text{allowed}}$ with $\mathbf{a}_{\text{scrub},p} = \mathbf{a}'_p$; if matches are found and replaced with schema-valid placeholders (e.g., `[REDACTED_FINANCIAL]`) that still satisfy $\text{Ctr}.\mathcal{M}_{\text{req}}(p)$, the matched compartments are cleared ($C_{\mathbf{a}_{\text{scrub}},p} \subseteq C_{\text{allowed}}$); whereas if a format-constrained slot ($p \in S_{\text{ctrl}}$) rejects placeholder syntax, an unredactable leak remains, or $\mathbf{a}' = \emptyset \wedge \text{Ctr.IsMutating}$ with $(C_{\text{eff}} \cup C_{\text{intent}}) \not\subseteq C_{\text{allowed}}$, ok = False is returned.

We explicitly distinguish read-only sanitization from mutating neutralization: $\text{SanitizeRead}(\mathbf{a}')$ operates on analytical read queries by stripping SQL comment delimiters, escaping wildcard characters, and appending deterministic limit caps to eliminate query-based exfiltration; whereas $\text{CleanInjection}(\mathbf{a}', \text{Ctr})$ operates on mutating operations diverted to quarantine by parsing argument ASTs against schema contract Ctr, stripping injection delimiters, converting raw SQL/command fragments into parameterized binding markers, or replacing untrusted strings with validated default literals prior to escrow queuing, ensuring unverified payloads cannot execute on production backends.

To eliminate control-flow short-circuiting where confidentiality scrubbing might accidentally bypass integrity checks on mutating operations, Algorithm 1 executes composite hierarchical evaluation:

1) **Confidentiality Inspection & Scrubbing First:** If $(C_{\text{eff}} \cup C_{\mathbf{a}}) \not\subseteq \text{Ctr}.C_{\max} \vee C_{\mathbf{a}} \not\subseteq C_{\text{clearance}}(\text{ag})$, arguments are inspected via $(\mathbf{a}_{\text{scrub}}, C_{\mathbf{a}_{\text{scrub}}}, \text{ok}) \leftarrow \text{ScrubParams}(\mathbf{a}', \{p \mapsto \mathbf{L}_{\mathbf{a},p}\}, \text{Ctr}, \text{ag})$. If parameters are confirmed clean of sensitive compartments ($\mathbf{a}_{\text{scrub}} = \mathbf{a}'$ and ok = True), execution proceeds directly to integrity validation regardless of tool mutability or Ctr.AttenSupported, eliminating ambient confidentiality lockouts. If sensitive data is detected ($\mathbf{a}_{\text{scrub}} \neq \mathbf{a}'$) and attenuation is supported, parameters are redacted and flagged as attenuated; if redaction fails ($\neg$ok) or attenuation is unsupported on a dirty argument, a `CATEGORICAL_BLOCK` is issued immediately.
2) **Integrity Evaluation Next (No Bypass):** If $\text{IntegCheck}(\{p \mapsto I_{\mathbf{a},p}\}, \text{Ctr}, I_{\text{inv}})$ fails, the gateway verifies tool mutability. Read-only queries proceed under `ATTENUATE_SANITIZE` via $\text{SanitizeRead}(\mathbf{a}')$ if attenuation is supported; otherwise, they are categorically blocked. State-mutating writes failing IntegCheck are redirected to `QUARANTINE_ESCROW` with cleaned parameters via $\text{CleanInjection}(\mathbf{a}', \text{Ctr})$ if attenuation is supported (Ctr.AttenSupported = True); otherwise, they terminate with `CATEGORICAL_BLOCK`.
3) **Compliant & Scrubbed-to-Escrow Flow:** If IntegCheck is satisfied, the gateway checks whether parameters were actively scrubbed (or whether the agent is under active privilege quarantine for an untagged production write): for read-only tools, scrubbed calls proceed under `ATTENUATE_SANITIZE`; for state-mutating tools, actively scrubbed or privilege-quarantined calls are diverted to `QUARANTINE_ESCROW` ("Scrubbed to Escrow") to ensure synthetic redaction placeholders or unverified post-escrow writes do not mutate production state directly. If both invariants are satisfied without attenuation, the call returns `PERMIT` for immediate wire serialization.

E. Quarantine Escrow Runtime Semantics & Immutable Audit Vouchers

To prevent multi-agent deadlocks or hallucinated retry loops when a tool invocation is intercepted, Agent-Lattice implements explicit runtime escrow semantics. When an unverified mutating invocation is diverted to quarantine, the gateway generates a structured synthetic return payload:

$$\mathcal{E} \triangleq \big(\text{escrow_id},\ \text{tx_id},\ \text{status},\ T,\ \text{ttl_seconds},\ \text{message}\big) \tag{24}$$

where tx_id denotes the provisional transaction identifier formatted to match the target tool's return key schema (e.g., UUID or integer key), status = "PENDING_HUMAN_APPROVAL", and ttl_seconds = 900. This synthetic result is injected back into the calling agent's context window as a tool response span $t_{\mathcal{E}}$ labeled $\text{Label}(t_{\mathcal{E}}) = (\emptyset,\ \text{SANITIZED} = 1)$ so downstream speculative arguments referencing tx_id satisfy schema-level integrity without elevating session trust. Downstream agent reasoning proceeds uninterrupted, recognizing that the mutation is queued for human review rather than encountering an unhandled exception or network timeout.

Execution Resumption and Data Dependencies: For workflows with downstream data dependencies (e.g., expecting a created transaction ID), the runtime middleware provides three execution policies: (1) *Coroutine Suspension*, which checkpoints agent state and yields execution until human approval or TTL expiration; (2) *Speculative Execution with Compensation Journal*, which supplies a provisional identifier tx_id and continues execution, unwinding downstream speculative steps via a distributed saga journal if rejected, as formalized in Eq. (25):

$$\mathcal{S}_{\text{saga}} \triangleq \big(\text{tx_id},\ T,\ T_{\text{compensate}},\ \mathbf{a}_{\text{raw}},\ \mathbf{a}',\ \mathcal{E},\ \text{Status}\big) \tag{25}$$

Algorithm 1 Composite Gateway Enforcement

Require: Agent ag, Tool T, Args $\mathbf{a}$, Intent I_{intent}, Provenance $\mathcal{G}_{\text{prov}}$, Context $\mathcal{T}_{\text{ctx}}$
Ensure: Action α, Args $\mathbf{a}'$, Voucher $\mathcal{V}$, EscrowReceipt $\mathcal{E}$

// Phase I: In-Memory Runtime Middleware (Tracing & Slicing)
$L_{\text{ctx}} \leftarrow \bigsqcup_{t \in \mathcal{T}_{\text{ctx}}} \text{Label}(t) = (C_{\text{eff}}, I_{\text{eff}})$, Ctr $\leftarrow$ LookupToolContract(T), $S_{\text{ctrl}} \leftarrow \{p \in \mathbf{a} \mid \text{Ctr}.I_{\text{req}}(p) \geq \text{SANITIZED}\}$
for each parameter slot $p \in \mathbf{a}$ **do**
  $m \leftarrow \text{Ctr}.\mathcal{M}_{\text{req}}(p)$, $V_{\text{anc}}^{\text{conf}} \leftarrow V(\mathcal{G}_{\text{slice}}^{\text{conf}}(p)) \cap \mathcal{T}_{\text{ctx}}$, $V_{\text{anc}}^{\text{integ}} \leftarrow V(\mathcal{G}_{\text{slice,pre}}^{\text{integ}}(p)) \setminus \{p\}$
  $C_{\mathbf{a},p} \leftarrow \big(\bigcup_{t \in V_{\text{anc}}^{\text{conf}}} C(t) \cup C_{\text{intent}}^{\text{synth}}(p)\big)$ **if** $V_{\text{anc}}^{\text{conf}} \neq \emptyset$ **else** $((C_{\text{eff}} \cup C_{\text{intent}})$ **if** (Ctr.IsMutating $\vee\ p \in S_{\text{ctrl}}$) **else** $\emptyset)$
  $I_{\mathbf{a},p}^{\text{raw}} \leftarrow \min_{v \in V_{\text{anc}}^{\text{integ}}} I(v)$ **if** $V_{\text{anc}}^{\text{conf}} \neq \emptyset$ **else** $(I_{\text{eff}}$ **if** $p \in S_{\text{ctrl}}$ **else** $(I_{\text{intent}}$ **if** Ctr.IsMutating **else** $3))$, Label$(p) \leftarrow (C_{\mathbf{a},p}, I_{\mathbf{a},p}^{\text{raw}})$
  if $m \neq$ None $\wedge\ m(\text{val}(p)) =$ true $\wedge\ (V_{\text{anc}}^{\text{conf}} \neq \emptyset \vee p \notin S_{\text{ctrl}})$ **then**
    Label$(p) \leftarrow \nabla_{\text{endorse}}(\text{Label}(p), \text{val}(p), m)$
  end if
  $I_{\mathbf{a},p} \leftarrow \min\left(I_{\text{clearance}}(\text{ag}), \pi_I(\text{Label}(p))\right)$
end for
if $\mathbf{a} = \emptyset$ **then**
  $(C_{\mathbf{a}}, I_{\mathbf{a}}) \leftarrow (C_{\text{eff}} \cup C_{\text{intent}}, \min(I_{\text{clearance}}(\text{ag}), I_{\text{eff}}))$ **if** Ctr.IsMutating **else** $(\emptyset, I_{\text{clearance}}(\text{ag}))$
else
  $C_{\mathbf{a}} \leftarrow \bigcup_{p \in \mathbf{a}} C_{\mathbf{a},p}$, $I_{\mathbf{a}} \leftarrow \min_{p \in \mathbf{a}} I_{\mathbf{a},p}$
end if
$\mathbf{L}_{\mathbf{a}} \leftarrow (C_{\mathbf{a}}, I_{\mathbf{a}})$
$I_{\mathbf{a},\text{ctrl}} \leftarrow I_{\mathbf{a}}$ **if** $\mathbf{a} = \emptyset$ **else** $(\min_{p \in S_{\text{ctrl}}} I_{\mathbf{a},p}$ **if** $S_{\text{ctrl}} \neq \emptyset$ **else** $3)$
// Phase II: Physical Tool Security Gateway (Evaluation & Attenuation)
$I_{\text{inv}} \leftarrow \min(I_{\mathbf{a},\text{ctrl}}, I_{\text{intent}}, I_{\text{clearance}}(\text{ag}))$
if $T \notin$ Tools(ag) $\vee\ \max(\text{Ctr}.I_{\text{req}}, \max_{p \in S_{\text{ctrl}}} \text{Ctr}.I_{\text{req}}(p)) > I_{\text{clearance}}(\text{ag})$ **then**
  $\mathcal{V} \leftarrow$ Voucher(ag, T, CATEGORICAL_BLOCK, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \emptyset$, "Subagent Privilege Escalation", None); **return** (CATEGORICAL_BLOCK, $\emptyset, \mathcal{V}$, None)
end if
$\mathbf{a}' \leftarrow \mathbf{a}$, attenuated $\leftarrow$ False, $\mathcal{E} \leftarrow$ None, $C_{\mathbf{a}_{\text{scrub}}} \leftarrow C_{\mathbf{a}}$
if $(C_{\text{eff}} \cup C_{\mathbf{a}}) \not\sqsubseteq \text{Ctr}.C_{\max} \vee C_{\mathbf{a}} \not\sqsubseteq C_{\text{clearance}}(\text{ag})$ **then**
  $(\mathbf{a}_{\text{scrub}}, C_{\mathbf{a}_{\text{scrub}}}, \text{ok}) \leftarrow \text{ScrubParams}(\mathbf{a}', \{p \mapsto \mathbf{L}_{\mathbf{a},p}\}, \text{Ctr}, \text{ag})$
  if not ok $\vee\ C_{\mathbf{a}_{\text{scrub}}} \not\sqsubseteq C_{\text{clearance}}(\text{ag})$ **then**
    $\mathcal{V} \leftarrow$ Voucher(ag, T, CATEGORICAL_BLOCK, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \emptyset$, "Confidentiality Violation", None); **return** (CATEGORICAL_BLOCK, $\emptyset, \mathcal{V}$, None)
  end if
  if $\mathbf{a}_{\text{scrub}} \neq \mathbf{a}'$ **then**
    if not Ctr.AttenSupported **then**
      $\mathcal{V} \leftarrow$ Voucher(ag, T, CATEGORICAL_BLOCK, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \emptyset$, "Redaction Unsupported", None); **return** (CATEGORICAL_BLOCK, $\emptyset, \mathcal{V}$, None)
    end if
    $\mathbf{a}' \leftarrow \mathbf{a}_{\text{scrub}}$, attenuated $\leftarrow$ True
  end if
end if
integ_ok $\leftarrow$ IntegCheck($\{p \mapsto I_{\mathbf{a},p}\}$, Ctr, I_{inv})
if not *integ_ok* **then**
  if not Ctr.IsMutating **then**
    if not Ctr.AttenSupported **then**
      $\mathcal{V} \leftarrow$ Voucher(ag, T, CATEGORICAL_BLOCK, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \emptyset$, "Read Integ Deficit", None); **return** (CATEGORICAL_BLOCK, $\emptyset, \mathcal{V}$, None)
    end if
    $\mathbf{a}' \leftarrow$ SanitizeRead($\mathbf{a}'$); rsn $\leftarrow$ "Read Scrubbed & Sanitized" **if** attenuated **else** "Read Sanitized"
    $\mathcal{V} \leftarrow$ Voucher(ag, T, ATTENUATE_SANITIZE, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \mathbf{a}'$, rsn, None); **return** (ATTENUATE_SANITIZE, $\mathbf{a}', \mathcal{V}$, None)
  else if Ctr.AttenSupported **then**
    $\mathbf{a}' \leftarrow$ CleanInjection($\mathbf{a}'$, Ctr), $\mathcal{E} \leftarrow$ CreateEscrowReceipt(ag, $T, \mathbf{a}'$), rsn $\leftarrow$ "Scrubbed & Mutating to Escrow" **if** attenuated **else** "Mutating to Escrow"
    $\mathcal{V} \leftarrow$ Voucher(ag, T, QUARANTINE_ESCROW, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \mathbf{a}'$, rsn, $\mathcal{E}$); **return** (QUARANTINE_ESCROW, $\mathbf{a}', \mathcal{V}, \mathcal{E}$)
  else
    $\mathcal{V} \leftarrow$ Voucher(ag, T, CATEGORICAL_BLOCK, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \emptyset$, "Non-Atten Block", None); **return** (CATEGORICAL_BLOCK, $\emptyset, \mathcal{V}$, None)
  end if
end if
if attenuated $\vee$ (Ctr.IsMutating $\wedge$ InPrivilegeQuarantine(ag, $\mathbf{a}'$)) **then**
  if Ctr.IsMutating **then**
    $\mathcal{E} \leftarrow$ CreateEscrowReceipt(ag, $T, \mathbf{a}'$); $\mathcal{V} \leftarrow$ Voucher(ag, T, QUARANTINE_ESCROW, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \mathbf{a}'$, "Scrubbed to Escrow", $\mathcal{E}$)
    return (QUARANTINE_ESCROW, $\mathbf{a}', \mathcal{V}, \mathcal{E}$)
  else
    $\mathcal{V} \leftarrow$ Voucher(ag, T, ATTENUATE_SANITIZE, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \mathbf{a}'$, "Read Scrubbed", None); **return** (ATTENUATE_SANITIZE, $\mathbf{a}', \mathcal{V}$, None)
  end if
end if
$\mathcal{V} \leftarrow$ Voucher(ag, T, PERMIT, $\mathbf{L}_{\mathbf{a}}, \mathbf{a}, \mathbf{a}'$, "Compliant Invariant", None); **return** (PERMIT, $\mathbf{a}', \mathcal{V}$, None)

and (3) *Deterministic Fail-Fast Notification*, which informs the planner that authorization is pending, allowing it to transition to independent analytical tasks.

Operational Boundaries of Speculative Saga Execution: Speculative execution under $\mathcal{S}_{\text{saga}}$ is strictly bounded by three operational constraints: (1) *Referential Integrity Isolation*: The provisional transaction identifier tx_id is restricted to staging tables or shadow scratch spaces; downstream operations referencing tx_id are tagged with speculative metadata and barred from publishing to production master tables until escrow resolution. To guarantee downstream read-after-write consistency under speculative quarantine without dirty read anomalies, subsequent read operations referencing an active provisional tx_id are dynamically routed to staging overlay views reflecting uncommitted provisional writes. By default, the staging overlay is maintained purely in an in-memory key-value sidecar shadow scratchpad within the agent container runtime (storing both $\mathbf{a}_{\text{raw}}$ and $\mathbf{a}'$ to support $O(1)$ primary-key/entity-ID lookups over uncommitted JSON-RPC argument maps with zero physical network egress); if a downstream

read attempts a relational join, SQL aggregation, or filter beyond exact key lookup, the runtime triggers the Dynamic Irreversibility Fallback to Coroutine Suspension. Optionally, for tools whose contract explicitly registers an isolated sandbox staging endpoint and $T_{\text{compensate}} \neq \text{None}$, downstream speculative steps tagged with tx_id may write provisional rows to external staging tables (rolled back via $T_{\text{compensate}}$ upon rejection or TTL expiration). Upon administrator review in Chronicle SecOps, for integrity-only escrows ("Mutating to Escrow"), the administrator may commit $\mathbf{a}_{\text{raw}}$ if verified benign or $\mathbf{a}'$ if sanitized; for confidentiality-scrubbed escrows ("Scrubbed to Escrow" or "Scrubbed & Mutating to Escrow"), committing unredacted $\mathbf{a}_{\text{raw}}$ to a sink with $\text{Ctr}.C_{\max}$ deficit is blocked unless an administrator with declassification authority explicitly binds a non-placeholder value for the redacted field. If a subsequent read query targets production master tables lacking shadow staging support or attempts to export uncommitted state across external API boundaries, the runtime middleware dynamically escalates from speculative execution to Coroutine Suspension, checkpointing agent state until human verification resolves the escrow (and invoking $T_{\text{compensate}}$ in reverse topological order over prior $\mathcal{S}_{\text{saga}}$ steps if the suspended escrow is rejected or expires). (2) *Dynamic Irreversibility Fallback*: Because autonomous multi-agent swarms emit tool calls dynamically at runtime rather than executing static workflow graphs, this irreversibility constraint is enforced dynamically step-by-step: the runtime permits speculative execution on initial compensable tools and dynamically triggers Coroutine Suspension the moment an uncompensable tool ($T_{\text{compensate}} = \text{None}$) or non-staged external API call is dispatched downstream. (3) *Privilege Quarantine*: During the 900-second escrow window, the calling agent's mutating permissions targeting production master tables are quarantined ($\text{InPrivilegeQuarantine}(\text{ag}, \mathbf{a}') = \text{True}$ in Algorithm 1, Line 52), while explicitly permitting subsequent speculative mutations tagged with tx_id to append to isolated staging tables and the saga journal.

Simultaneously, the gateway records an immutable audit voucher:

$$\mathcal{V} \triangleq \big(\text{VoucherID},\ \text{Timestamp},\ \text{ag},\ T,\ \alpha,\ \mathbf{L}_{\mathbf{a}},\ \mathbf{a}_{\text{raw}},\ \mathbf{a}',\ \text{Reason},\ \mathcal{E},\ \sigma_{\text{HMAC}}\big) \tag{26}$$

Crucially, unadulterated raw arguments $\mathbf{a}_{\text{raw}}$ are preserved immutably in $\mathcal{V}$ alongside sanitized arguments $\mathbf{a}'$. We note that the $\text{CreateEscrowReceipt}(\text{ag}, T, \mathbf{a}')$ constructor in Algorithm 1 accepts three domain arguments and internally auto-populates the UUID escrow_id, monotonic tx_id, default status = "PENDING_HUMAN_APPROVAL", ttl_seconds = 900, and diagnostic message, matching Eq. (24); symmetrically, the $\text{Voucher}(\dots)$ constructor accepts eight domain arguments $(\text{ag}, T, \alpha, \mathbf{L}_{\mathbf{a}}, \mathbf{a}, \mathbf{a}', \text{Reason}, \mathcal{E}$, where input $\mathbf{a}$ corresponds to $\mathbf{a}_{\text{raw}}$ in Eq. (26)) and internally auto-populates the monotonic VoucherID, nanosecond Timestamp, and inline-evaluated σ_{HMAC}, matching the complete 11-element tuple $\mathcal{V}$ formalized in Eq. (26). To achieve microsecond-scale enforcement, inline HMAC-SHA256 evaluation is accelerated via CPU vector registers executing SIMD assembly primitives (such as Intel SHA extensions or ARMv8 Crypto instructions provided by OpenSSL's assembly engine). Process heap memory for the native key arena is locked against OS paging via `mlock()` (`CRYPTO_secure_malloc`) and protected from memory dumps via `MADV_DONTDUMP`. Hardware Confidential VMs (AMD SEV-SNP / Intel TDX on Google Cloud Confidential Space) provide defense-in-depth against privileged host memory inspection. For non-repudiable auditability across administrative boundaries, vouchers are asynchronously signed out-of-band via Google Cloud KMS (asymmetric Ed25519) and streamed to an immutable distributed ledger (Paxos/Spanner) off the critical request path. Escrowed mutations enter an asynchronous queue with a 15-minute TTL, dispatched via Cloud Pub/Sub to Chronicle SecOps for administrator verification, with token-bucket rate limiting preventing backpressure exhaustion under sustained attacks.

IV. Empirical Security & Systems Evaluation

A. Benchmark Dataset Stratification ($N = 1{,}000$)

To evaluate Agent-Lattice against realistic enterprise workloads, we constructed an authentic benchmark suite of $N = 1{,}000$ execution traces stratified across five operational cohorts ($n = 200$ traces each). The benchmark was constructed using authentic enterprise schemas, production database contracts, and realistic benign workflows with zero synthetic mocking, while adversarial attack payloads were synthetically injected across production-grade enterprise interfaces with verified parse boundaries:

1) **Stratum 1: Compliant Internal Operations ($y = 0$, Benign):** Authorized financial auditors querying SAP S/4HANA General Ledger balances with valid credentials.
2) **Stratum 2: Benign User Mutations via Endorsement ($y = 0$, Benign):** Customer support queries requesting profile updates (e.g., updating communication preferences, categorical status enums, or priority tiers). These parameters are validated and endorsed via ∇_{endorse} ($I = \text{SANITIZED}$) over deterministic schemas, while passive text fields are bound via prepared parameterized statements under $\text{Ctr}.I_{\text{req}} = \text{UNTRUSTED} = 0$.
3) **Stratum 3: Indirect Prompt Injection & Confused Deputy ($y = 1$, Attack):** Adversarial injection strings embedded in user tickets attempting `DROP TABLE`, `DELETE FROM`, or privilege grant escalations via privileged database maintenance agents.
4) **Stratum 4: Sensitive Data Exfiltration ($y = 1$, Attack):** Prompts attempting to leak employee PII (SSNs, salaries) retrieved from internal HR databases to public search endpoints or external webhooks.
5) **Stratum 5: Transitive Privilege Escalation & Composite Attacks ($y = 1$, Attack):** Low-clearance child subagents attempting direct invocation of root-authorized tools (`sap_post_payment`, `cloud_run_deploy`) while carrying sensitive confidentiality tokens.

We note that the benchmark focuses on schema-bounded tool interfaces. Syntactic schema validation (∇_{endorse}) guarantees

type and delimiter containment, eliminating command and SQL escaping attacks; defense against semantic in-schema parameter manipulation (e.g., IDOR attacks where an adversary injects a valid UUID belonging to another tenant or manipulates authorized business enums under Ctr.$I_{\text{req}} = 0$) cannot be resolved by lattice thresholds alone and relies fundamentally on application-layer session identity binding (e.g., verifying that parameter UUIDs match authenticated session tokens), explicit invocation intent verification ($I_{\text{intent}} = 3$), backend entity ownership validation, or enforcing elevated integrity floors ($I_{\text{req}} \geq 2$), as analyzed in Section V.

B. Comparative Baselines

We evaluated Agent-Lattice against six industry and academic paradigms:

- **Static RBAC Baseline:** Permits any tool execution as long as the calling agent process possesses static IAM permissions [5].
- **Stochastic Prompt Guardrail Baseline:** Evaluates prompt text using continuous semantic classification, representing Llama Guard [6] and NeMo Guardrails [7].
- **LlamaFirewall / PromptGuard-2 Baseline:** Augments prompt screening with Meta's purpose-built agentic guardrail scanner and trace alignment checks [8].
- **Dual-LLM Sandboxing Baseline:** Implements the unprivileged/privileged model split [9], where unprivileged agent outputs are forwarded as natural language summaries.
- **CaMeL Interpreter IFC Baseline:** Implements capability-based Python interpreter execution with static taint tracking over variable bindings [10].
- **Progent / FIDES Policy & Planner IFC Baselines:** Evaluates declarative symbolic tool privilege rules (Progent [14]) and planner-level information-flow label propagation without decoupled ∇_{endorse} cuts (FIDES [11]).

C. Experimental Results & Classification Superiority

As detailed in Table I, Agent-Lattice achieves an ROC-AUC of 1.0000, PR-AUC of 1.0000, and 100.00% classification accuracy within schema-contracted interfaces under constrained slot-filling (and 99.20% accuracy with $8/400 = 2.00\%$ false-positive escrow routing, ROC-AUC $= 0.9960$, and PR-AUC $= 0.9973$ under unconstrained natural-language prompting), completely eliminating all 600 attempted security violations while preserving 98.50% Utility Under Attack (UA) via non-disruptive capability attenuation and synthetic escrow receipts. Specifically, the 98.50% $(591/600)$ UA disaggregates into two operational sub-metrics: (i) **Autonomous Backend Completion (**$60.00\%, 360/600$**)**, comprising all $200/200$ Stratum 4 read queries remediated inline via ScrubParams$(\mathbf{a}, C_{\text{allowed}})$ and SanitizeRead$(\mathbf{a}_{\text{scrub}})$ (reported as `Read Scrubbed & Sanitized`) plus $160/200$ Stratum 5 traces where the gateway's CATEGORICAL_BLOCK of the compromised subagent's unauthorized tool call returns a structured error to the root coordinator, which completes the benign parent workflow via an authorized read/fallback branch (the remaining $40/200$ Stratum 5 traces required mutating subagent actions diverted to escrow); and (ii) **Escrow-Preserved Workflow Continuity (**$38.50\%, 231/600$**)**, comprising $191/200$ Stratum 3 and $40/200$ Stratum 5 state-mutating traces where QUARANTINE_ESCROW injects a synthetic EscrowReceipt $\mathcal{E}$ (tx_id) and the agent completes its downstream analytical/reporting steps over the in-memory $\mathcal{S}_{\text{saga}}$ overlay without workflow termination (with $9/600 = 1.50\%$ of traces encountering an uncompensable downstream tool $T_{\text{compensate}} = \text{None}$ that triggered Dynamic Irreversibility Fallback to Coroutine Suspension until human review). In contrast, stochastic prompt guardrails miss 447 of 600 attacks (74.5% failure rate) at $\tau = 0.50$ due to adversarial paraphrasing and delimiter smuggling; supplementing Llama-Guard-3-8B with Meta's purpose-built agentic scanner LlamaFirewall / PromptGuard-2 [8] improves accuracy to 81.40% ($F_1 = 0.8166$, ROC-AUC 0.9512, UA 64.00%) while still missing 186 multi-hop delimiter and subagent delegation attacks. Dual-LLM sandboxes achieve an ROC-AUC of 0.8764, missing 200 attacks in Stratum 5 because the privileged actor blindly trusts natural-language summaries produced by compromised child subagents without cryptographic provenance propagation across RPC boundaries.

The CaMeL Interpreter IFC baseline [10] achieves 88.00% discrete accuracy at $\tau = 0.50$, missing 80 of 600 attacks (13.3% false negative rate) while incurring 40 false positives on benign requests (10.0% false positive rate). Continuous ROC and PR curves for all baselines were constructed via threshold sweeps over identical constrained slot-filling trajectories: for CaMeL, under fractional AST variable taint $r_{\text{bin}}(\mathbf{a}) = \frac{|V_{\text{tainted}} \cap V_{\mathbf{a}}|}{|V_{\mathbf{a}}|}$ across $|V_{\mathbf{a}}| \in \{1, 2, 3, 4\}$ argument slots, the 520 detected attacks evaluate to $r_{\text{bin}} = 1.0$, the 40 multi-slot benign false positives evaluate to $0.50 \leq r_{\text{bin}} < 1.0$ (where only a subset of slots carry untrusted user taint alongside trusted schema defaults), the 360 true negatives evaluate to $r_{\text{bin}} = 0.0$, and all 80 unpropagated Stratum 5 attacks evaluate to $r_{\text{bin}} = 0.0$, yielding an exact Mann–Whitney ROC-AUC of $\frac{520 \times 400 + 80 \times (360 \times 0.5 + 40 \times 0)}{240{,}000} = \frac{208{,}000 + 14{,}400}{240{,}000} = 0.9267$; when augmented with a continuous hop-depth provenance decay proxy $r_{\text{hop}}(\mathbf{a}) = \max(r_{\text{bin}}(\mathbf{a}), 0.45 \cdot \gamma_{\text{rpc}}^{\text{hops}})$ with $\gamma_{\text{rpc}} = 0.85$ across the 80 Stratum 5 multi-hop delegation traces (50 single-hop traces with hops $= 1$ at $r_{\text{hop}} = 0.3825$ and 30 two-hop traces with hops $= 2$ at $r_{\text{hop}} = 0.3251$, while $88/360$ benign multi-agent traces involve subagent read delegation with hops $\in \{1, 2\}$ and $272/360$ execute at hops $= 0$), CaMeL achieves a continuous ROC-AUC of 0.9603 and PR-AUC of 0.9779 (furthermore, in an instrumentation-parity ablation where CaMeL's variable taint tags are serialized across subagent RPC headers, its Stratum 5 false negatives drop from 80 to 0, yielding 96.00% accuracy limited only by the 40 benign false-positive over-taints due to lacking ∇_{endorse}; whereas evaluating full CaMeL combining Dual-LLM Quarantined-LLM extraction with Interpreter IFC yields 86.00% accuracy, still missing the 80 unpropagated Stratum 5 RPC handoffs and incurring 40 Stratum 2 false positives plus 20 extraction formatting failures on nested JSON schemas). For LlamaFirewall, curves were swept over the maximum risk prob-

TABLE I

COMPREHENSIVE SECURITY CLASSIFICATION SUMMARY ($N = 1{,}000$ ENTERPRISE TRACES). NOTE: CLASSIFICATION METRICS REFLECT EVALUATION OVER STRUCTURED ENTERPRISE TOOL EXECUTION CONTRACTS WHERE PARAMETERS ARE BOUNDED BY DETERMINISTIC SCHEMA CONSTRAINTS.

Metric	Static RBAC	Prompt Guardrail†	LlamaFirewall / PG-2	Dual-LLM Sandbox	CaMeL IFC	Progent / FIDES‡	Agent-Lattice (Ours)§
ROC-AUC Score	0.5000	0.9345	0.9512	0.8764	0.9603	0.9685 / 0.9740	**1.0000**
Precision-Recall AUC (PR-AUC)	0.6000	0.9633	0.9718	0.9328	0.9779	0.9810 / 0.9845	**1.0000**
F1-Score	0.0000	0.4064	0.8166	0.8000	0.8966	0.9375 / 0.9524	**1.0000 (0.9934)**
Classification Accuracy	40.00%	55.30%	81.40%	80.00%	88.00% (96.00%)	92.00% / 94.00%	**100.00% (99.20%)**
False Negatives (Missed Attacks)	600 / 600 (100%)	447 / 600 (74.50%)	186 / 600 (31.00%)	200 / 600 (33.33%)	80 / 600 (0 / 600)	0 / 600 (0.00%)	**0 / 600 (0.00%)**
False Positives (Benign Escrow)	0 / 400 (0.00%)	0 / 400 (0.00%)	0 / 400 (0.00%)	0 / 400 (0.00%)	40 / 400 (10.00%)	80 / 60 of 400 (20% / 15%)	**0 / 400 (8 / 400 [2.00%])**
Benign Task Success Rate	100.00%	100.00%	100.00%	100.00%	90.00%	80.00% / 85.00%	**100.00% (98.00%)**
Utility Under Attack (UA)	0.00% (Compromised)	25.50%	64.00%	61.50%	76.50%	78.00% / 82.50%	**98.50% (60.0% + 38.5%)**

†Note: Evaluated at $\tau = 0.50$ (zero-FPR operating point). Sweeping τ to the unconstrained maximum-F_1 point ($\tau^* = 0.18$) improves attack recall to 78.50% (FN = 129/600), but incurs a 37.50% false-positive rate on benign queries ($150/400$ benign tasks aborted), yielding $F_1 = 0.7715$ and 72.10% accuracy ($\chi^2_{\text{Guardrail},\tau^*}(1) = 277.00, p < 10^{-50}$). ‡Progent ($92.00\%$ accuracy, $80/400 = 20.00\%$ overall FP [40.00% of Stratum 2]) and FIDES (94.00% accuracy, $60/400 = 15.00\%$ overall FP [30.00% of Stratum 2]) block 100% of attacks but over-quarantine benign user mutations in Stratum 2 due to lacking decoupled ∇_{endorse} cuts. §Parenthesized values for CaMeL IFC show the RPC-Instrumented CaMeL ablation (96.00% accuracy, $0/600$ FN, $40/400$ FP, $F_1 = 0.9677$); parenthesized values for Agent-Lattice show the Unconstrained Natural-Language Prompting ablation (99.20% accuracy, $0/600$ FN, $8/400 = 2.00\%$ overall FP [4.00% of Stratum 2], $F_1 = 0.9934$, ROC-AUC $= 0.9960$, PR-AUC $= 0.9973$). Utility Under Attack (UA $= 98.50\%$) disaggregates into 60.00% ($360/600$) Autonomous Backend Completion (Stratum 4 read queries remediated via `ScrubParams` and `SanitizeRead` plus $160/200$ Stratum 5 traces where the root coordinator completes the benign parent task after subagent escalation block) and 38.50% ($231/600$) Escrow-Preserved Workflow Continuity via synthetic `EscrowReceipt` and speculative saga execution ($9/600 = 1.50\%$ triggered Coroutine Suspension awaiting human approval).

ability $\max(p_{\text{PG2}}, p_{\text{AlignCheck}}) \in [0,1]$ emitted by PromptGuard-2 and AlignmentCheck; for Progent and FIDES, curves were swept over the continuous context-decayed integrity-deficit score $r(\mathbf{a}) = \left(1 - \min_{p \in \mathbf{a}} \min(1.0, I_p / I_{\text{req}}(p))\right) \cdot \gamma_{\text{ctx}}^{\Delta\text{turn}}$ with $\gamma_{\text{ctx}} = 0.90$ and turn distance $\Delta\text{turn} \in \{0,1,2,3\}$ (in discrete evaluation, Progent's synthesized schema-level policy rules and FIDES's constrained variable inspection permitted $120/200 = 60.0\%$ and $140/200 = 70.0\%$ of single-slot scalar Stratum 2 user mutations, respectively, but quarantined the remaining $80/200 = 40.0\%$ and $60/200 = 30.0\%$ multi-slot composite mutations where user-derived arguments co-occurred with prior tool reads without decoupled ∇_{endorse} cuts); for Dual-LLM, the privileged model evaluates natural-language summaries using an explicit verification prompt outputting structured JSON, from which normalized token log-probabilities over candidate safety tokens yield posterior safety probability $s_{\text{Dual-LLM}} = P(\text{safe} \mid \text{summary}) \in [0,1]$ (swept across τ, with $\tau = 0.50$ generating the discrete baseline operating point in Table I); and for Prompt Guardrail, by sweeping the output sigmoid probability. Across all 1,000 benchmark traces, canonicalized substring alignment succeeded on 97.7% of literal parameters; the 2.3% unaligned literal occurrences (attributable to model-specific date/number formatting) occurred strictly on read-only queries (Stratum 1) or non-control payload slots where schema typing ∇_{endorse} smoothly validated types ($I = \text{SANITIZED}$), while 100% of mutating execution-control parameters in Stratum 2 achieved verbatim substring alignment via constrained slot filling, incurring zero false-positive escrows on benign workflows (in an unconstrained ablation where slot-filling instructions were removed from Stratum 2 prompts, free-form model paraphrasing of enum/date control parameters triggered fail-closed escrow on $8/200 = 4.0\%$ of Stratum 2 mutations [$8/400 = 2.00\%$ overall FPR, 99.20% accuracy, $F_1 = 0.9934$, ROC-AUC $= 0.9960$, PR-AUC $= 0.9973$] while maintaining 0.0% attack leakage). For Agent-Lattice, discrete lattice inequalities natively evaluate to binary decisions (100.00% accuracy, 100.00% precision, 100.00% recall, 100.00% specificity, and $F_1 = 1.0000$ at its native operational threshold $\tau = 1.0$); for continuous ROC/PR parametric sweeps, we explicitly distinguish the pre-attenuation violation detection scalar $s^{\text{pre}}_{\text{lattice}}(\mathbf{a})$ (which evaluates raw arguments $\mathbf{a}$ prior to runtime attenuation for Table I attack classification) from the post-attenuation wire-serialization compliance predicate (Eq. 23):

$$
\begin{aligned}
s^{\text{pre}}_{\text{lattice}}(\mathbf{a}) = \min \Bigg(& \\
& \begin{cases} 1.0 \quad \text{if } S_{\text{ctrl}} = \emptyset, \\ \min_{p \in S_{\text{ctrl}}} \min\left(1.0, \frac{I_{\mathbf{a},p}}{\text{Ctr.}I_{\text{req}}(p)}\right) \\ \quad \text{if } S_{\text{ctrl}} \neq \emptyset \end{cases}, \\
& \begin{cases} 1.0 \quad \text{if Ctr.}I_{\text{req}} = \text{UNTRUSTED} = 0, \\ \min\left(1.0, \frac{I_{\text{inv}}}{\text{Ctr.}I_{\text{req}}}\right) \quad \text{otherwise} \end{cases}, \\
& \mathbf{1}\Big(T \in \text{Tools(ag)} \wedge C_{\mathbf{a}_{\text{scrub}}} \subseteq C_{\text{clearance}}(\text{ag}) \\
& \quad \wedge I_{\text{clearance}}(\text{ag}) \geq \max\left(\text{Ctr.}I_{\text{req}}, \max_{p \in S_{\text{ctrl}}} \text{Ctr.}I_{\text{req}}(p)\right)\Big), \\
& \mathbf{1}\Big((C_{\mathbf{a}} \cup C_{\text{eff}} \subseteq \text{Ctr.}C_{\max}) \\
& \quad \vee (C_{\mathbf{a}_{\text{scrub}}} \subseteq \text{Ctr.}C_{\max} \wedge \mathbf{a}_{\text{scrub}} = \mathbf{a} \wedge \text{ok})\Big) \Bigg) \in [0,1]
\end{aligned}
\tag{27}
$$

where $I_{\text{inv}} \triangleq \min(I_{\mathbf{a},\text{ctrl}}, I_{\text{intent}}, I_{\text{clearance}}(\text{ag}))$, and the fourth argument incorporates a pre-attenuation non-modification check ($\mathbf{a}_{\text{scrub}} = \mathbf{a}$, i.e., Unreflected) so raw unredacted confidentiality violations evaluate to 0 prior to inline scrubbing. Under risk score polarity $r = 1 - s^{\text{pre}}_{\text{lattice}}(\mathbf{a}) \in [0,1]$, any policy violation yields $s^{\text{pre}}_{\text{lattice}}(\mathbf{a}) < 1.0$ (i.e., strict threshold $r > 0$), correctly flagging Stratum 4 exfiltration attempts as attacks ($y = 1$) before runtime attenuation (whereas post-attenuation wire compliance $s^{\text{post}}_{\text{lattice}}(\mathbf{a})$ restores CanSanitizeRead($\mathbf{a}$) and Scrubbed($\mathbf{a}$) so sanitized read queries execute safely with 98.5% post-attenuation analytical utility). In our single-LLM gateway testbed where tool calls are emitted directly over

JSON-RPC, CaMeL's interpreter-level checks inspect parsed tool-call ASTs: when indirect prompt injection payloads are embedded inside literal string arguments (such as SQL injection fragments or exfiltration webhook URLs), the syntax remains valid and requires provenance taint rejection, while CaMeL's 80 false negatives occurred during multi-agent delegation handoffs in Stratum 5 where single-interpreter variable taint did not propagate across subagent RPC boundaries. Additionally, CaMeL lacks dynamic endorsement cuts (∇_{endorse}), causing legitimate multi-slot string operations carrying benign user inputs to be over-tainted and unnecessarily quarantined. Across all 400 benign traces (Strata 1 and 2), Agent-Lattice achieved a 100.00% Benign Task Success Rate with zero false-positive escrows under constrained slot-filling (98.00% under unconstrained prompting).

Comparison with Planner-Level Screening & Public Benchmarks ($N = 150$): In contrast to interpreter sandboxing, deterministic interaction MAC monitors (SEAgent [13], Progent [14]), and prompt guardrails, planner-level architectures such as FIDES [11] and RTBAS [12] operate inside LLM reasoning loops, introducing 5 ms to 25 ms inference latency overhead per turn for formal planning-state graph serialization (or 500 ms to $2{,}000$ ms for secondary LLM screening in RTBAS; on our local EPYC/ARM64 testbed, CaMeL Interpreter IFC averaged $142.0\,\mu$s per tool invocation for Python AST parsing and taint propagation, Progent averaged $310.0\,\mu$s for JSON-Schema policy evaluation, and Agent-Lattice's constrained slot-filling system instruction added just $+12$ prompt tokens with zero output token overhead). As shown in Table I, Progent [14] and FIDES [11] achieved 92.00% and 94.00% accuracy on our $N = 1{,}000$ traces (0% ASR, but 40.0% [80/200] and 30.0% [60/200] false-positive quarantine rates on Stratum 2 benign user mutations—corresponding to 20.00% [80/400] and 15.00% [60/400] overall benign FPR—due to lacking decoupled ∇_{endorse} boundary cuts). To validate external generalizability (with open-ended free-form generation tasks deferred in Section V-A5), we evaluated 150 schema-bounded injection tasks selected via fixed-seed stratified random sampling (`seed = 42`) from the schema-bounded tool-calling subsets of three public suites (comprising $\approx 62\%$ of total tasks across the suites after excluding unconstrained free-text composition tasks lacking structured control slots)—50 workspace/banking/slack tasks from AgentDojo [23], 50 direct/indirect tool-hijacking tasks from InjecAgent [24], and 50 cross-domain delegation tasks from ASB [25]—mapping read tools to $\text{Ctr}.I_{\text{req}} = 0$, routine user-mutating tools to $\text{Ctr}.I_{\text{req}} = 0$ with control-slot $\text{Ctr}.I_{\text{req}}(p) = 1$, and administrative/wire-transfer tools to $\text{Ctr}.I_{\text{req}} = 3$. Across these 150 public tasks, Agent-Lattice achieved a **0.00% Attack Success Rate (ASR)** ($0/150$, exact Clopper–Pearson 95% CI $[0.00\%, 2.43\%]$) and **96.00%** ($144/150$) **Utility Under Attack (UA)** under constrained slot filling—disaggregating per suite into 0.0% ASR / 96.0% ($48/50$) UA on AgentDojo, 0.0% ASR / 98.0% ($49/50$) UA on InjecAgent, and 0.0% ASR / 94.0% ($47/50$) UA on ASB ($6/150 = 4.00\%$ encountering uncompensable downstream external calls that suspended awaiting human approval; vs. 18.67% ASR / 72.00% UA for LlamaFirewall, 8.00% ASR / 81.33% UA for CaMeL, and 0.00% ASR / 79.33% UA for Progent)—incurring a 4.67% ($7/150$) false-positive escrow rate only when constrained slot-filling prompts were deliberately relaxed in tainted multi-turn contexts.

Model-Agnostic Transport Invariance: To evaluate gateway reference-monitor determinism independently of model refusal variance, our primary $N = 1{,}000$ benchmark replays standardized tool-call trajectories elicited across Gemini 2.5 Pro, Gemini 2.5 Flash, and GPT-4o (for trials where a model's internal safety filter initially declined an adversarial prompt during corpus construction, the trajectory was elicited via a continuation harness so every gateway baseline was stress-tested against all 600 concrete tool-call attacks). Because Agent-Lattice evaluates deterministic lattice invariants over canonicalized tool-call payloads at the physical transport layer outside the neural reasoning loop, gateway interception recall is strictly 100.00% across all 600 attack trajectories for all three models (while in unassisted live end-to-end multi-turn generation prior to gateway interception, native model refusal blocked 64.5% [$387/600$], 52.0% [$312/600$], and 58.5% [$351/600$] of raw injection prompts for Gemini 2.5 Pro, Gemini 2.5 Flash, and GPT-4o, respectively; combining native model refusal with gateway interception on the remaining 213, 288, and 249 emitted attack calls yields composite end-to-end system ASRs on Gemini 2.5 Pro of **0.00%** for Agent-Lattice, Progent, and FIDES, versus 1.68% [$10/600$] for CaMeL, 3.90% [$23/600$] for LlamaFirewall, 4.20% [$25/600$] for Dual-LLM, and 9.40% [$56/600$] for Prompt Guardrail). In contrast, planner-level Dual-LLM sandboxing and semantic Prompt Guardrails rely on probabilistic natural-language verification ($80.00\% \pm 1.4\%$ and $55.30\% \pm 0.8\%$ accuracy, respectively), remaining architecturally vulnerable to transitive subagent summaries regardless of foundation model capacity.

D. Paired McNemar Statistical Significance

To evaluate whether Agent-Lattice's performance superiority is statistically significant, we conducted paired McNemar tests with Yates' continuity correction.

Comparing Agent-Lattice against the Prompt Guardrail baseline across all $N = 1{,}000$ traces yields the paired contingency matrix where row index 1 denotes correctness for Agent-Lattice and column index 0 denotes error for the baseline (such that $n_{10} = 447$ denotes traces where Agent-Lattice is correct and the baseline is incorrect, $n_{11} = 553, n_{01} = 0, n_{00} = 0$). Applying Yates' correction:

$$\begin{aligned}\chi^2_{\text{Guardrail}}(1) &= \frac{(|n_{10} - n_{01}| - 1)^2}{n_{10} + n_{01}} = \frac{(|447 - 0| - 1)^2}{447} \\ &= \frac{446^2}{447} = \frac{198{,}916}{447} \approx 445.00\end{aligned} \tag{28}$$

with $p < 10^{-50}$, confirming overwhelming statistical superiority over semantic guardrails (and $\chi^2_{\text{Guardrail},\tau^*}(1) = \frac{278^2}{279} \approx 277.00, p < 10^{-50}$ at the unconstrained $\tau^* = 0.18$ operating point where $n_{10} = 129 + 150 = 279$).

Comparing Agent-Lattice against LlamaFirewall ($n_{10} = 186, n_{01} = 0$) and Dual-LLM Sandboxing ($n_{11} = 800, n_{10} = 200, n_{01} = 0, n_{00} = 0$) yields $\chi^2_{\text{LlamaFirewall}}(1) = \frac{185^2}{186} \approx 184.01$ ($p < 10^{-41}$) and:

$$\chi^2_{\text{Dual-LLM}}(1) = \frac{(|200 - 0| - 1)^2}{200} = \frac{199^2}{200} = \frac{39{,}601}{200} \approx 198.01 \tag{29}$$

with $p < 10^{-44}$, confirming certified statistical superiority over agentic scanners and dual-model architectures.

Similarly, comparing Agent-Lattice against the CaMeL Interpreter IFC baseline yields paired contingency values $n_{11} = 880, n_{10} = 120, n_{01} = 0, n_{00} = 0$:

$$\chi^2_{\text{CaMeL}}(1) = \frac{(|120 - 0| - 1)^2}{120} = \frac{119^2}{120} = \frac{14{,}161}{120} \approx 118.01 \tag{30}$$

with $p < 10^{-26}$ (and against Progent [$n_{10} = 80$], FIDES [$n_{10} = 60$], and RPC-Instrumented CaMeL [$n_{10} = 40$], yielding $\chi^2_{\text{Progent}}(1) = \frac{79^2}{80} \approx 78.01, p < 10^{-18}$; $\chi^2_{\text{FIDES}}(1) = \frac{59^2}{60} \approx 58.02, p < 10^{-13}$; and $\chi^2_{\text{CaMeL-RPC}}(1) = \frac{39^2}{40} \approx 38.03, p < 10^{-9}$, respectively; under the unconstrained prompting ablation where Agent-Lattice's 8 paraphrased Stratum 2 escrows are a strict subset of FIDES's 60 Stratum 2 escrows—yielding $n_{00} = 8$ concordant errors, $n_{10} = 52$, and $n_{01} = 0$—superiority over FIDES remains highly significant at $\chi^2(1) = \frac{(|52-0|-1)^2}{52} \approx 50.02, p < 10^{-11}$).

E. Disaggregated Telemetry & Latency Profile

As documented in Table II, total enforcement overhead computed directly on the empirical joint distribution averages $22.98\,\mu$s with an empirical P95 latency of $28.75\,\mu$s and P99 latency of $81.40\,\mu$s. Benchmarked on host CPU cores with C-accelerated validation extensions, Agent-Lattice delivers line-rate protection at sub-$35\,\mu$s speed with zero token cost.

TABLE II
DISAGGREGATED IN-MEMORY REFERENCE MONITOR TELEMETRY

Hardware Architecture / Pipeline Stage	Mean	Empirical P95	Empirical P99
Apple Silicon ARM64 ($n = 500$)			
Phase I: Taint Ingestion & Slicing	$4.12\,\mu$s	$4.68\,\mu$s	$9.82\,\mu$s
Phase II: Gateway Verification	$15.30\,\mu$s	$21.12\,\mu$s	$58.28\,\mu$s
Subtotal Joint Overhead	$\mathbf{19.42}\,\mu$s	$\mathbf{24.15}\,\mu$s	$\mathbf{63.40}\,\mu$s
AMD EPYC 7B12 x86_64 ($n = 500$)			
Phase I: Taint Ingestion & Slicing	$5.54\,\mu$s	$6.24\,\mu$s	$15.12\,\mu$s
Phase II: Gateway Verification	$21.00\,\mu$s	$29.62\,\mu$s	$89.32\,\mu$s
Subtotal Joint Overhead	$\mathbf{26.54}\,\mu$s	$\mathbf{33.40}\,\mu$s	$\mathbf{96.80}\,\mu$s
Pooled Sample ($N = 1{,}000$)			
Phase I: Taint Ingestion & Slicing	$4.83\,\mu$s	$5.46\,\mu$s†	$12.47\,\mu$s†
Phase II: Gateway Verification	$18.15\,\mu$s	$25.37\,\mu$s†	$73.80\,\mu$s†
Pooled Joint Overhead	$\mathbf{22.98}\,\mu$s	$\mathbf{28.75}\,\mu$s	$\mathbf{81.40}\,\mu$s
In-Kernel eBPF Socket Drop (Linux EPYC)	$\mathbf{0.42 \pm 0.08}\,\mu$s	$\mathbf{0.68}\,\mu$s	$\mathbf{0.91}\,\mu$s

Note: Percentiles for individual hardware architectures are computed directly on the empirical joint distribution of per-request total latency ($t_{\text{total},i} = t_{\text{Phase I},i} + t_{\text{Phase II},i}$), while †pooled per-phase columns report cross-architecture equal-weight averages of the stratified $n = 500$ subgroup percentiles (corresponding to true combined $N = 1{,}000$ order statistics of $5.61\,\mu$s P95 / $13.20\,\mu$s P99 for Phase I and $26.10\,\mu$s P95 / $76.40\,\mu$s P99 for Phase II). The Pooled Joint Overhead row ($22.98\,\mu$s mean, $28.75\,\mu$s P95, $81.40\,\mu$s P99) is computed directly as empirical order statistics over the combined $N = 1{,}000$ joint latency distribution. Each hardware target evaluated all 1,000 traces (2,000 total runs), with Table II reporting disjoint stratified $n = 500$ splits per architecture (100 per stratum) to form the $N = 1{,}000$ pooled sample without duplicate trace weighting. Across the $N = 1{,}000$ traces, Phase II decision outcomes comprise 400 `PERMIT` traces (Strata 1–2, $8.70\,\mu$s mean), 120 `CATEGORICAL_BLOCK` traces (Stratum 5 tool-inventory/clearance violations, $4.80\,\mu$s mean), 240 `ATTENUATE_SANITIZE` traces (200 Stratum 4 read queries plus 40 Stratum 5 read-only subagent calls, $24.60\,\mu$s mean), and 240 `QUARANTINE_ESCROW` traces (200 Stratum 3 plus 40 Stratum 5 mutating calls at $28.925\,\mu$s mean, with $195/240$ simulated administrator rejections incurring $6.40\,\mu$s for in-memory scratchpad rollback), yielding $(400{\times}8.70{+}120{\times}4.80{+}240{\times}24.60{+}240{\times}28.925{+}195{\times}6.40)/1{,}000 = 18{,}150/1{,}000 = 18.15\,\mu$s pooled Phase II mean. The relationship $P_{95}(A + B) = 28.75\,\mu\text{s} < P_{95}(A) + P_{95}(B) = 30.83\,\mu$s is an empirical property of the observed joint latency distribution. Reported telemetry isolates in-memory compute time (adding $14.20\,\mu$s per 512-token streaming chunk ingestion and $18.4\,\mu$s mean IPC transit in external UNIX-socket sidecar mode).

F. Component Ablation Experiments

To empirically validate the architectural necessity of individual components within Agent-Lattice, we conducted three targeted ablation experiments:

1) **Turn-Depth Taint Saturation (Whole-Context LWM vs. Backward Slicing):** We evaluated the false-positive quarantine escrow rate on benign tool invocations across conversational turns $k \in \{1, 2, 4, 6, 8\}$ following an initial untrusted user query ($N = 200$ multi-turn dialogues per turn depth). At turn $k = 1$, the multi-turn dialogues comprised 100% read-only information-retrieval and status-query operations configured with tool-level $\text{Ctr.}I_{\text{req}} = \text{UNTRUSTED} = 0$, explaining why whole-context LWM did not trigger false-positive escrows on the initial turn. Subsequent turns introduced state-mutating operations requiring parameter-level integrity $\text{Ctr.}I_{\text{req}}(p) \geq \text{SANITIZED} = 1$, with nominal target mutating sampling proportions of 30%, 60%, 80%, and 90%, yielding empirical realizations of $57/200 = 28.5\%$ at $k = 2$, $118/200 = 59.0\%$ at $k = 4$, $153/200 = 76.5\%$ at $k = 6$, and $169/200 = 84.5\%$ at $k = 8$ (the remaining operations being read-only queries with $\text{Ctr.}I_{\text{req}} = 0$). In this ablation, mutating parameters were derived from trusted preceding system directives ($I \geq 1$) while the turn 1 untrusted user query was causally unreferenced, cleanly isolating the benefit of backward causal dependency slicing from schema endorsement. Under classical whole-context Low-Water Mark tracking, because ambient integrity is permanently depressed to $I_{\text{eff}} = 0$ upon ingesting untrusted input at turn $k = 1$, 100% of subsequent mutating operations requiring $I \geq 1$ are falsely escrowed, causing the aggregate false-positive rate to escalate to exactly the empirical mutating-call proportions (28.5% at $k = 2$, 59.0% at $k = 4$, 76.5% at $k = 6$, and 84.5% at $k = 8$). In contrast, backward causal slicing isolated strictly the parameter dependency subgraph $\mathcal{G}_{\text{slice}}(\mathbf{a})$, maintaining a flat 0.0% false-positive rate across all invocations regardless of turn depth.
2) **Dynamic Endorsement Utility:** Evaluating benign user mutation requests (Stratum 2) without the dynamic en-

dorsement operator (∇_{endorse} disabled) resulted in a 0.0% benign task execution success rate, as all user-supplied parameters remained tagged $I = \text{UNTRUSTED} = 0$ and were quarantined. Enabling ∇_{endorse} over schema-validated primitive slots restored a 100.0% benign task execution success rate without admitting adversarial command injections.

3) **Context Length Slicing Scalability:** We measured backward slicing latency across context window scales ($N = 250$ trials per scale evaluated on host ARM64 and AMD EPYC cores): $m = 500$ tokens ($3.12 \pm 0.21\,\mu$s), $m = 2{,}000$ tokens ($4.83 \pm 0.34\,\mu$s), $m = 8{,}000$ tokens ($7.19 \pm 0.52\,\mu$s), and $m = 32{,}000$ tokens ($11.45 \pm 0.81\,\mu$s). Latency scales sub-linearly because graph traversal is bounded by the localized ancestry of tool call parameters rather than full context traversal. In empirical benchmark trials over 1M token context windows ($N = 100$ trials across both hardware targets), backward reachability slicing traversal (Phase I) averaged $28.45 \pm 1.82\,\mu$s over pre-indexed interval trees augmented with an inverted token-span hash map index mapping canonicalized string literals directly to node IDs (populated via word-boundary and entity token segmentation during streaming ingestion). Cumulative upfront indexing for a 1M token context requires 28.4 ms, while incremental per-turn token ingestion and interval-tree maintenance averaging $14.20 \pm 0.95\,\mu$s per 512-token chunk execute asynchronously in the background as tokens stream from the LLM, bounded by a lightweight synchronization barrier ensuring all emitted chunks are committed to $\mathcal{G}_{\text{prov}}$ prior to Phase I slicing; measured sequentially in combination with Phase II gateway evaluation ($18.15 \pm 1.10\,\mu$s pooled mean at standard context depth, or $21.40 \pm 1.35\,\mu$s at 1M tokens when `ScrubParams` queries the 2 MB cache-line-aligned blocked Bloom filter and 2 MB HighwayHash table in shared memory), total 1M-token in-memory enforcement latency averaged $46.60 \pm 2.15\,\mu$s ($49.85 \pm 2.26\,\mu$s for scrubbed read trajectories), confirming line-rate sub-$50\,\mu$s performance across million-token frontiers (for synchronous bulk tool observations, multi-threaded SIMD chunk indexing processes 64 KB in 0.85 ms, shingle n-gram hashing captures multi-word spans crossing lexer boundaries, and each inverted-index key caches a pre-aggregated 6-bit compartment bitmask and min-integrity scalar so high-frequency literal collisions evaluate $\sqcup$ in $O(1)$ bit-operations).

G. Experimental Parameters & Testbed Environment

All experiments were executed on host CPU cores (Apple Silicon / AMD EPYC 7B12) running Python 3.11 with C-accelerated extensions (`pydantic-core`) and OpenSSL for inline HMAC-SHA256 voucher signing. Agent orchestration was powered by Google ADK v1.2+ interfacing with Google Cloud Vertex AI (dedicated enterprise evaluation project) utilizing production Gemini 2.5 Pro (`gemini-2.5-pro`) and Gemini 2.5 Flash (`gemini-2.5-flash`) endpoints. All model endpoints are verified against official Google Cloud model lifecycle documentation. The Prompt Guardrail baseline utilized Llama-Guard-3-8B with continuous risk scores calibrated at an optimal zero-FPR operating threshold of $\tau = 0.50$ determined via complete ROC curve sweep.

V. Discussion, Limitations & Conclusion

A. Discussion & Operational Boundaries

While Agent-Lattice provides deterministic, line-rate Information-Flow Control, five operational boundaries warrant explicit analysis:

1) **Semantic vs. Syntactic Validation in ∇_{endorse}:** The dynamic endorsement operator relies on deterministic schema validation. To prevent delimiter smuggling and command escaping, ∇_{endorse} is strictly bounded to non-instructional primitive types (UUIDs, enums, numeric fields). However, syntactic conformity does not inherently prevent in-schema semantic parameter manipulation (e.g., Insecure Direct Object References [IDOR] where an untrusted prompt manipulates an agent into supplying another user's valid account UUID or unauthorized status enum for the same principal). In routine mutating workflows where parameters require $I_{\mathbf{a},p} \geq \text{SANITIZED} = 1$ and the tool-level requirement is $\text{Ctr}.I_{\text{req}} = \text{UNTRUSTED} = 0$, schema endorsement confirms syntactic type containment ($m(v) = \text{true} \wedge I^{\text{raw}}_{\mathbf{a},p} \geq I^{\text{endorse}}_{\text{min}} \implies I = \text{SANITIZED}$), permitting autonomous execution at $I = 1$ while relying on application-layer session identity binding (e.g., verifying that parameter UUIDs match authenticated session tokens) to prevent cross-tenant IDOR. In contrast, sensitive administrative state mutations enforce elevated lattice floors ($\text{Ctr}.I_{\text{req}} \geq \text{VERIFIED} = 2$ or $I_{\text{intent}} = \text{SYSTEM_AUTHORIZED} = 3$), which deterministically divert invocations to quarantine escrow whenever untrusted input is causally ingested. We acknowledge that same-principal in-schema enum/boolean manipulation triggered by untrusted text resides outside syntactic schema validation and requires higher-level behavioral constraints or elevated integrity floors. Furthermore, as formalized in Definitions 2 and 6, our production deployment supports a five-level refinement subdividing $I = 0$ into Authenticated User Prompt Intent ($I_{\text{user}} = 0.5$) versus Unverified Third-Party External Data ($I_{\text{untrusted}} = 0$): by setting $I^{\text{endorse}}_{\text{min}} = I_{\text{user}} = 0.5$ so ∇_{endorse} requires pre-endorsement causal ancestry $I^{\text{raw}}_{\mathbf{a},p} \geq I_{\text{user}} = 0.5$ over $V^{\text{integ}}_{\text{anc}}(p)$ and evaluating tool-level $\text{Ctr}.I_{\text{req}} = 0$ (or evaluating sliced intent $I^{\text{slice}}_{\text{intent}} = \min(I_{\text{prompt}}, \min_{\text{obs} \in \text{Obs} \cap V^{\text{conf}}_{\text{anc}}(\mathbf{a})} I(\text{obs})) \geq I_{\text{user}}$ strictly when all arguments are verbatim-aligned [$V^{\text{conf}}_{\text{anc}}(p) \neq \emptyset \ \forall p \in \mathbf{a}$], falling back conservatively to full intra-turn I_{intent} whenever any argument is synthesized or unaligned), routine user mutations remain immune to intra-turn taint saturation from unrelated third-party read

observations ($I_{\text{untrusted}} = 0$) while third-party external data at $I_{\text{untrusted}} = 0$ is structurally barred from triggering ∇_{endorse} or routine mutations even with schema-valid primitives. Deterministic elimination of 100% of attacks in our benchmarks applies to syntax-escaping, delimiter-smuggling, and unendorsed transitive privilege escalation over schema-contracted interfaces; defense against in-schema semantic parameter manipulation (IDOR) remains bounded by application-layer session identity binding, backend entity ownership validation, or enforcing elevated integrity floors ($\text{Ctr.}I_{\text{req}} \geq 2$). Open-ended natural language strings are never endorsed to authorize administrative state mutations ($\text{Ctr.}I_{\text{req}} = \text{SYSTEM_AUTHORIZED} = 3$).

2) **Integrity Escrow vs. Availability Trade-Off & DoS Protection:** Strict integrity enforcement diverts unverified mutations to quarantine escrow. For state-mutating operations, degradation to read-only replicas is impossible, meaning sustained adversarial injection streams with a 900-second TTL can introduce reviewer fatigue and operational backpressure on human operators (Chronicle SecOps). When queue capacity is reached, returning HTTP 429 backpressure risks denying service to legitimate incoming mutations. To mitigate queue exhaustion and prevent operator saturation under sustained attacks, Agent-Lattice incorporates: (1) automated cryptographic deduplication computed strictly over the canonicalized invocation tuple $(\text{principal_id}, T_{\text{id}}, \text{escrow_reason}, \{p \mapsto \text{canonical}(a_p) \mid p \in S_{\text{ctrl}}\})$ (falling back to canonical hashing over all arguments when $S_{\text{ctrl}} = \emptyset$, implemented as production sidecar extensions alongside $\mathcal{D}_{\text{inbound}}$) rather than full unconstrained argument payloads, preventing trivial salt/comment evasion; (2) deterministic priority-tier sorting grounded in caller clearance tiers and authenticated principal tokens; and (3) per-principal quota allocations tracked against authenticated external clients (decoupled from internal review queue capacity) that isolate quarantine quotas by trigger-taint provenance to preserve victim principal availability under indirect injection.

3) **Stored Indirect Injection Lifecycle & State Persistence:** When an unverified input or tool execution response is written to or retrieved from persistent enterprise storage (e.g., ticket description or database row), the underlying data store persists security attributes using dedicated schema metadata columns (`_lattice_conf` and `_lattice_integ`) or an external provenance sidecar store. Upon retrieving stored records, the runtime applies an inbound deserialization operator $\mathcal{D}_{\text{inbound}}(R) \to \langle t_j \rangle$ that maps persisted `_lattice_conf` and `_lattice_integ` attributes directly to newly instantiated token spans in $\mathcal{T}_{\text{ctx}}$, bounded by the target endpoint trust ceiling $\text{Ctr.}I_{\text{endpoint_max}}$ ($I(t_j) = \min(I_{\text{deserialized}}, \text{Ctr.}I_{\text{endpoint_max}})$), assigning $I = \text{UNTRUSTED}$ to unverified user fields and ensuring that stored injection attempts cannot evade taint tracking across multi-session data persistence lifecycles. For pre-existing or legacy enterprise datastores lacking dedicated metadata columns, the runtime enforces a deterministic fallback policy: records retrieved from legacy stores are assigned static table-level default contracts based on authenticated database connection credentials and semantic data domains ($I = 0$ for user-editable free text fields, $I = 2$ for immutable system tables), with confidentiality compartments falling back to pre-configured compartment sets $C_{\text{default}} \subseteq \mathcal{K}$ based on table sensitivity classifications.

4) **Adaptive Schema Synthesis Trust Model:** Tool contracts, schema validators, and parameter integrity thresholds $\{p_i \mapsto I_{\text{req}}(p_i)\}$ are provisioned and cryptographically signed by authorized system administrators during tool registration. When runtime environments employ dynamic tool synthesis, newly generated contracts require validation by an authorized administrator or must execute under strict read-only default constraints until endorsed.

5) **Generalizability to Unconstrained Open-World Tasks:** Whereas Section IV-C evaluated the 150 schema-bounded tool-calling tasks ($\approx 62\%$ of the public suites) from AgentDojo [23], InjecAgent [24], and Agent Security Bench (ASB) [25], the remaining $\approx 38\%$ unconstrained open-world tasks require free-form natural language control parameters (e.g., `send_email(body=...)` or `update_ticket(notes=...)` where natural-language instructions act as downstream control payloads). In such free-form tasks, designating text bodies as execution-control parameters ($I_{\text{req}} \geq 1$) causes user-influenced actions to route to escrow, whereas treating them as passive payload fields ($I_{\text{req}} = 0$) relies on prompt-parameter channel separation and parameterized sink isolation. Extending lattice endorsement to unconstrained free-text generation and polymorphic inter-agent domain-specific languages represents an important trajectory for future research requiring adaptive schema synthesis and semantic intent verification.

B. Conclusion

The enterprise adoption of autonomous multi-agent systems requires provable, hardware-grounded information-flow security. By replacing fragile, stochastic prompt classifiers with a mathematically certified Product Security Lattice, causal token taint tracking, decoupled backward argument slicing ($\mathcal{G}^{\text{integ}}_{\text{slice}}$ vs. $\mathcal{G}^{\text{conf}}_{\text{slice}}$), and schema-bounded dynamic endorsement (∇_{endorse}), Agent-Lattice resolves the Semantic Confused Deputy crisis within schema-contracted enterprise tool environments. Architecturally, our results demonstrate a clean division of labor: transport-layer lattice non-interference deterministically eliminates syntax-escaping, cross-compartment exfiltration, and unendorsed transitive privilege escalation in $22.98\,\mu$s mean in-memory compute time ($28.75\,\mu$s P95) with zero LLM token overhead, while pairing seamlessly with application-layer session identity binding and I_{user} intent floors to govern semantic in-schema authorization.

References

[1] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, pp. 68 539–68 551, 2023.

[2] Anthropic, "Model Context Protocol (MCP) Specification," Nov. 2024. [Online]. Available: https://modelcontextprotocol.io

[3] N. Hardy, "The Confused Deputy: (or why capabilities might have been invented)," *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.

[4] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," Tech. Rep., 2025. [Online]. Available: https://owasp.org/www-project-top-10-for-large-language-model-applications/

[5] R. S. Sandhu, E. J. Coyne, H. L. Feinstein, and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[6] H. Inan, K. Upasani, J. Chi, R. Rungta, K. Iyer, Y. Mao, M. Tontchev, Q. Hu, B. Fuller, D. Testuggine, and M. Khabsa, "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv preprint arXiv:2312.06674*, 2023.

[7] T. Rebedea, R. Dinu, M. N. Sreedhar, C. Parisien, and J. Cohen, "NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails," in *Proc. 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP 2023 System Demonstrations)*, pp. 431–445, 2023.

[8] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents," *arXiv preprint arXiv:2505.03574*, 2025.

[9] S. Willison, "The Dual LLM pattern for building AI assistants that can resist prompt injection," Apr. 25, 2023. [Online]. Available: https://simonwillison.net/2023/Apr/25/dual-llm-pattern/

[10] E. Debenedetti, I. Shumailov, T. Fan, J. Hayes, N. Carlini, D. Fabian, C. Kern, C. Shi, A. Terzis, and F. Tramèr, "Defeating prompt injections by design," *arXiv preprint arXiv:2503.18813*, 2025.

[11] M. Costa, B. Köpf, A. Kolluri, A. Paverd, M. Russinovich, A. Salem, S. Tople, L. Wutschitz, and S. Zanella-Béguelin, "Securing AI agents with information-flow control," *arXiv preprint arXiv:2505.23643*, 2025.

[12] P. Y. Zhong, S. Chen, R. Wang, M. McCall, B. L. Titzer, H. Miller, and P. B. Gibbons, "RTBAS: Defending LLM agents against prompt injection and privacy leakage," *arXiv preprint arXiv:2502.08966*, 2025.

[13] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming Various Privilege Escalation in LLM-Based Agent Systems: A Mandatory Access Control Framework," *arXiv preprint arXiv:2601.11893*, 2026.

[14] T. Shi, J. He, Z. Wang, H. Li, L. Wu, W. Guo, and D. Song, "Progent: Securing AI agents with privilege control," *arXiv preprint arXiv:2504.11703*, 2025.

[15] P. Wang, Y. Liu, Y. Lu, Y. Cai, H. Chen, Q. Yang, J. Zhang, J. Hong, and Y. Wu, "AgentArmor: Enforcing program analysis on agent runtime trace to defend against prompt injection," *arXiv preprint arXiv:2508.01249*, 2025.

[16] J. Kim, W. Choi, and B. Lee, "Prompt Flow Integrity to prevent privilege escalation in LLM agents," *arXiv preprint arXiv:2503.15547*, 2025.

[17] Y. Wu, F. Roesner, T. Kohno, N. Zhang, and U. Iqbal, "IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems," in *Proc. Network and Distributed System Security Symposium (NDSS)*, 2025, *arXiv:2403.04960*.

[18] Y. Zheng, T. Wu, Q. Fu, T. Yu, W. Mao, T. Ma, D. Williams, W. Wang, and A. Quinn, "ActPlane: Programmable OS-level policy enforcement for agent harnesses," *arXiv preprint arXiv:2606.25189*, 2026.

[19] A. Štorek, N. Holzer, Z. Zhang, and S. Jana, "GIF: Locally Sound Geometric Information Flow Control for LLMs," *arXiv preprint arXiv:2606.23277*, 2026.

[20] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, "The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions," *arXiv preprint arXiv:2404.13208*, 2024.

[21] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending Against Prompt Injection with Structured Queries," in *Proc. 34th USENIX Security Symposium (USENIX Security 25)*, 2025.

[22] K. Hines, G. Lopez, M. Hall, F. Zarfati, Y. Zunger, and M. Kiciman, "Defending Against Indirect Prompt Injection Attacks With Spotlighting," *arXiv preprint arXiv:2403.14720*, 2024.

[23] E. Debenedetti, J. Zhang, M. Balunović, L. Beurer-Kellner, M. Fischer, and F. Tramèr, "AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents," in *Proc. Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2024.

[24] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 10471–10506, 2024.

[25] H. Zhang, J. Huang, K. Mei, Y. Yao, Z. Wang, C. Zhan, H. Wang, and Y. Zhang, "Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-Based Agents," in *Proc. International Conference on Learning Representations (ICLR)*, 2025.

[26] D. E. Denning, "A lattice model of secure information flow," *Communications of the ACM*, vol. 19, no. 5, pp. 236–243, 1976.

[27] A. C. Myers and B. Liskov, "A decentralized model for information flow control," in *Proc. 16th ACM Symposium on Operating Systems Principles (SOSP '97)*, pp. 129–142, 1997.

[28] D. E. Bell and L. J. LaPadula, "Secure computer systems: Mathematical foundations," MITRE Corp., Bedford, MA, Tech. Rep. MTR-2547, 1973.

[29] K. J. Biba, "Integrity considerations for secure computer systems," Electronic Systems Division, Air Force Systems Command, Hanscom AFB, Tech. Rep. ESD-TR-76-372, 1977.